# Approximate (Neural) Q-Learning

## Abstract

This topic delves into approximate solution methods in reinforcement learning, focusing on deep Q-learning and neural network function approximation. It builds on the understanding of Q-learning and addresses the challenges of high-dimensional state spaces through Deep Q-Networks (DQN). We cover the progression from tabular Q-learning through neural Q-learning to the full DQN algorithm with experience replay, target networks, frame stacking, and other stabilization techniques. With this, students will understand how modern deep reinforcement learning tackles complex problems like playing Atari games from raw pixels.

## Learning Objectives

- Understand the limitations of tabular Q-learning and the need for function approximation
- Implement neural Q-learning with deep neural networks
- Apply DQN stabilization techniques: experience replay, target networks, frame stacking
- Train deep RL agents on high-dimensional visual observation spaces

# Literature

This lecture draws from the following sources:

**Primary Sources:**

**Sutton & Barto Textbook:**


- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Online preprint (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)
  - Chapters 9-10: On-policy Prediction and Control with Function Approximation
  - Section 11.3: The "Deadly Triad" (function approximation + bootstrapping + off-policy learning)


**Deep Q-Networks (DQN):**


- Mnih, V., et al. (2015). "Human-level control through deep reinforcement learning." *Nature*, 518(7540), 529-533. PDF (https://web.stanford.edu/class/psych209/Readings/MnihEtAlHassibis15NatureControlDeepRL
- Eysenbach, B. (2025). "Lecture 11: Deep Q Learning." *COS435/ECE 433: Introduction to RL*. PDF (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf)
- Levine, S. (2023). "Lecture 8: Deep Reinforcement Learning with Q-Functions." *CS285: Deep RL*. PDF (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf)

# Motivation & Gap Analysis

**What We Know from Previous Lectures:**

- **L1-L2:** Core RL concepts (MDPs, Bellman equations, policies, value functions)
- **L3:** Policy gradient methods for direct policy optimization
- **L4:** Dynamic programming for exact solutions in known environments
- **L5:** Monte Carlo methods for learning from complete episodes
- **L6:** Temporal-Difference learning (SARSA, Q-learning) for efficient online learning

**The Challenge We Face:** All methods covered so far rely on **tabular representations** where we explicitly store $Q(s, a)$ or $V(s)$ for every state-action pair or state. This approach has severe limitations:

1. **Curse of Dimensionality:** Real-world problems have enormous or continuous state spaces (e.g., robot joint angles, pixel values from images). Storing a table for billions of states is computationally infeasible.
2. **No Generalization:** Tabular methods cannot generalize from similar states. If you learn $Q(\text{robot at position A}, \text{move right})$, this tells you nothing about $Q(\text{robot at position B}, \text{move right})$, even if A and B are nearly identical.
3. **Sample Inefficiency:** You must visit every state-action pair many times to learn accurate values. In high-dimensional spaces, this requires an astronomical number of samples.

**Solution: Function Approximation**

Instead of storing values in tables, we approximate value functions using **parameterized functions** (neural networks):

- $Q(s, a) \approx Q_\theta(s, a)$ where $\theta$ are network parameters
- $V(s) \approx V_\phi(s)$ where $\phi$ are network parameters

This enables:

- **Scalability:** Neural networks can handle high-dimensional inputs (images, continuous states)
- **Generalization:** Similar states produce similar values automatically
- **Efficiency:** Learning from one state improves value estimates for similar states

**New Challenges This Lecture Addresses:**

1. **Stability:** Combining function approximation with TD learning and off-policy methods creates the "deadly triad" that can cause divergence. We address this with:
    - **Experience Replay:** Breaking correlations in sequential data
    - **Target Networks:** Stabilizing the learning target
2. **Visual Observations:** Learning directly from high-dimensional pixel inputs requires:
    - **Convolutional Neural Networks:** Efficient processing of spatial information
    - **Frame Stacking:** Capturing temporal dynamics and motion
    - **Frame Skipping:** Computational efficiency and action persistence

**What This Lecture Enables:** By the end, you will be able to apply RL to complex, high-dimensional problems like:

- Playing Atari games from raw pixels (DQN)
- Any domain where tabular methods fail due to state space size
- Visual control tasks requiring convolutional neural networks

This lecture bridges the gap from classical RL (suitable for toy problems) to deep RL (suitable for real-world applications).

# Approximate Neural Q-Learning

## Q-Learning Overview

**Q-Learning: Off-Policy TD Control**

Learn optimal action-value function $Q^*(s, a)$ directly from experience, without needing environment model.

([Levine et al., CS285 Lecture 8, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf))

**Update Rule:**

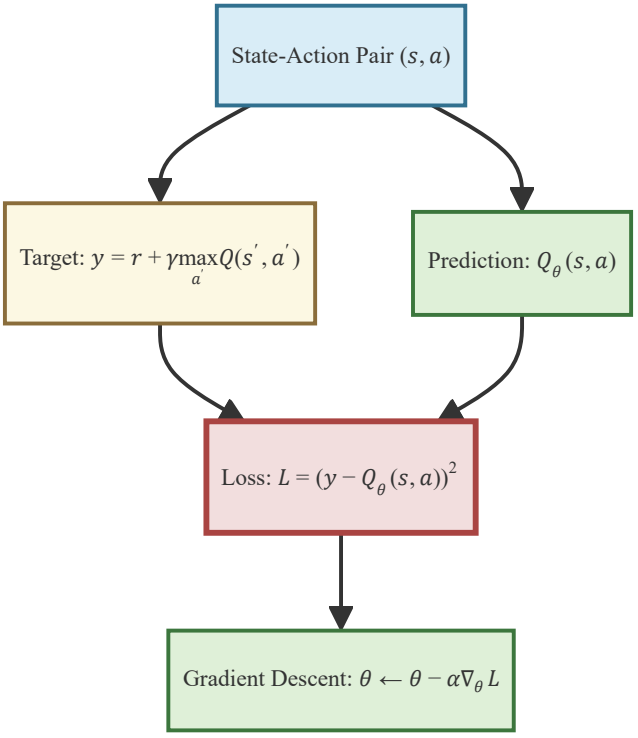
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

**Key Properties:**

- **Off-policy**: Can learn from any policy (e.g., $\epsilon$-greedy)
- **Model-free**: No environment dynamics needed
- **Convergence**: Guaranteed under tabular case with appropriate conditions
- **Target**: Learns $Q^*$ (optimal), not $Q^\pi$ (policy-specific)

([Eysenbach, COS435 Lecture 11: Deep Q Learning, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf))

## From Q-Learning to Supervised Learning




### TD Error Minimization as Regression

The Q-learning update can be reframed as a **supervised learning problem**:

([Levine et al., CS285 Lecture 8, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf))

**Traditional Q-Learning Update:**

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

**Reframed as Supervised Learning:**

1. **Input**: State-action pair $(s, a)$
2. **Target Label**: $y = r + \gamma \max_{a'} Q(s', a')$ (TD target)
3. **Prediction**: $\hat{y} = Q_\theta(s, a)$ (network output)
4. **Loss Function**: $\mathcal{L}(\theta) = \frac{1}{2}(y - Q_\theta(s, a))^2$ (MSE)
5. **Optimization**: Minimize loss via gradient descent

**Key Insight:**

> Minimizing the TD error $\delta = y - Q(s, a)$ is mathematically equivalent to performing regression where the Q-network learns to predict the TD target.

**Why This Matters:**

- **Leverage supervised learning tools**: We can use standard gradient-based optimization (SGD, Adam)
- **Use neural networks**: Any differentiable function approximator works
- **Batch training**: Can train on mini-batches of experiences
- **Established infrastructure**: PyTorch, TensorFlow provide all needed tools

**From Tabular to Deep:**

| Aspect | Tabular Q-Learning | Deep Q-Learning |
|---|---|---|
| Update | Direct value update | Gradient descent on loss |
| Representation | Table $Q[s, a]$ | Neural network $Q_\theta(s, a)$ |
| Target | $r + \gamma \max_{a'} Q(s', a')$ | Same TD target |
| Optimization | TD update rule | Backpropagation + SGD |

This reformulation is the **foundation of DQN**: treating RL as supervised learning with dynamically generated labels.

## Adapting Q-Learning for Neural Network Approximation

To apply Q-Learning with a neural network instead of a table, we need minimal adaptations:

### Key Adaptations

1. **State Representation**:
   - Tabular: State index $s \in \{0, 1, \ldots, 15\}$ directly indexes Q-table
   - Neural Network: Convert state index to **one-hot encoding** $\mathbf{s} \in \mathbb{R}^{16}$
   - Example: State 3 → `[0, 0, 0, 1, 0, ..., 0]`
   - Why? Representing the state with a single number would imply that there is a natural ordering over the states. This is not the case for gridworld environments.
2. **Q-Value Update**:
   - Tabular: `Q[s, a] ← Q[s, a] + α[R + γ max Q[s', a'] - Q[s, a]]`
   - Neural Network: Minimize loss $\mathcal{L} = (Q_\theta(s, a) - [R + \gamma \max_{a'} Q_\theta(s', a')])^2$
   - The gradient descent step replaces the direct table update

## Pseudocode: Q-Learning with Neural Network

**Q-Learning with Function Approximation (Online)**
Input: Neural network $Q_\theta(s, a)$ with parameters $\theta$, step size $\alpha$, discount $\gamma$, exploration $\epsilon$
Output: Learned $Q_\theta$ approximating $Q^*$

**Algorithm:**
**1.** Initialize $Q_\theta$ with random weights

**2. Loop for each episode:**
  **3.** Initialize state $s$

  **4. Loop for each step of episode:**
    **5.** Choose $a$ from $s$ using $\epsilon$-greedy policy derived from $Q_\theta$
    **6.** Take action $a$, observe $r, s'$

    **7.** Compute TD target: $y \leftarrow r + \gamma \max_{a'} Q_\theta(s', a')$
    **8.** Compute prediction: $\hat{y} \leftarrow Q_\theta(s, a)$
    **9.** Compute loss: $\mathcal{L} \leftarrow (\hat{y} - y)^2$

    **10.** Update network parameters: $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$

    **11.** $s \leftarrow s'$
  **Until** $s$ is terminal

In [1]:
```
# Q-Learning with Neural Network Function Approximation
# Based on Sutton & Barto, Section 6.5 (Q-Learning) with neural n
etwork replacing Q-table
# Follows exact structure from L6 Q-learning, using NN instead of
table

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym
import importlib
import sys
sys.path.append('.')
import helper_functions
importlib.reload(helper_functions)
from helper_functions import (state_to_onehot, epsilon_greedy_pol
icy,
                                get_q_values_for_visualization, pl
ot_q_values_and_learning_curves)
import time

# Neural network definition
class QNetwork(nn.Module):
    """Q-Network: Neural network for Q-value approximation"""
    def __init__(self, n_states, n_actions, hidden_sizes=[64, 6
4]):
        super(QNetwork, self).__init__()
        layers = []
        input_size = n_states
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(input_size, hidden_size))
            layers.append(nn.ReLU())
            input_size = hidden_size
        layers.append(nn.Linear(input_size, n_actions))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

def q_learning_neural_network(env_name='FrozenLake-v1',
                                max_total_steps=10000,
                                alpha=0.001,
                                gamma=0.99,
                                epsilon=0.1,
                                hidden_sizes=[64, 64],
                                show_animation=False,
                                plot_interval=1000,
                                render_interval=None,
                                max_episode_steps=100):
    """
    Q-Learning with Neural Network Function Approximation.
    Follows exact structure of tabular Q-learning from L6, Sectio
n 6.5, p. 131

    Args:
```

```
        env_name: Gymnasium environment name
        max_total_steps: Maximum total steps across all episodes
        alpha: Step size (learning rate for optimizer)
        gamma: Discount factor
        epsilon: Small ε > 0 for ε-greedy behavior policy
        hidden_sizes: List of hidden layer sizes
        show_animation: If True, show learning progress
        plot_interval: Update visualization every N steps
        render_interval: Render environment animation every N ste
ps (None = no rendering)
        max_episode_steps: Maximum steps per episode

    Returns:
        Q_net: Learned Q-network
        policy: Learned policy (greedy w.r.t. Q)
        stats: Dictionary with learning statistics
    """
    # Create training environment
    env = gym.make(env_name, is_slippery=False, render_mode=None,
                   max_episode_steps=max_episode_steps)

    # Create separate rendering environment if needed
    env_render = None
    if render_interval is not None:
        if 'CartPole' in env_name:
            env_render = gym.make(env_name, render_mode='human')
        else:
            env_render = gym.make(env_name, is_slippery=False, re
nder_mode='human',
                                  max_episode_steps=max_episode_st
eps)

    # Step 1: Initialize Q_ϑ(s,a) - neural network with random we
ights
    num_states = env.observation_space.n
    num_actions = env.action_space.n

    # Set random seeds
    np.random.seed(0)
    torch.manual_seed(0)

    # Initialize Q-network
    Q_net = QNetwork(num_states, num_actions, hidden_sizes=hidden
_sizes)

    # Optimizer and loss (for gradient-based update of Q-network)
    optimizer = optim.Adam(Q_net.parameters(), lr=alpha)
    criterion = nn.MSELoss()

    # Statistics tracking
    stats = {
        'episode_returns': [],
        'episode_lengths': [],
        'losses': [],
        'cumulative_steps': []
    }
```

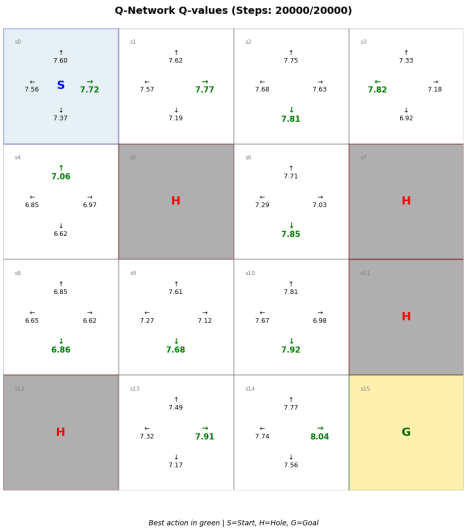
```
    # Show initial Q-values
    if show_animation:
        Q_init = get_q_values_for_visualization(Q_net, num_state
s, num_actions)
        plot_q_values_and_learning_curves(Q_init, stats, 0, max_t
otal_steps)
        time.sleep(5)

    # Step 2: Loop for episodes until max_total_steps reached
    total_steps = 0
    episode = 0

    while total_steps < max_total_steps:
        # Use render environment at intervals
        use_render = (render_interval is not None and
                      total_steps % render_interval < max_episode_
steps)
        current_env = env_render if use_render else env

        # Step 3: Initialize S
        state, _ = current_env.reset()

        episode_reward = 0
        episode_steps = 0
        done = False
        episode_losses = []

        # Step 4: Loop for each step of episode
        while not done and episode_steps < max_episode_steps and
total_steps < max_total_steps:
            # Step 5: Choose A from S using policy derived from Q
(ε-greedy)
            action = epsilon_greedy_policy(Q_net, state, num_stat
es, num_actions, epsilon)

            # Step 6: Take action A, observe R, S'
            next_state, reward, done, truncated, info = current_e
nv.step(action)
            episode_reward += reward
            episode_steps += 1
            total_steps += 1

            # Step 7: Q(S,A) ← Q(S,A) + α[R + γ max_a Q(S',a) - Q
(S,A)]
            # Neural network version: minimize loss = (Q_ϑ(S,A) -
[R + γ max_a Q_ϑ(S',a)])²

            # Convert states to one-hot encoding
            state_onehot = torch.tensor(state_to_onehot(state, nu
m_states),
                                        dtype=torch.float32).unsqu
eeze(0)
            next_state_onehot = torch.tensor(state_to_onehot(next
_state, num_states),
                                             dtype=torch.float32).
unsqueeze(0)
```

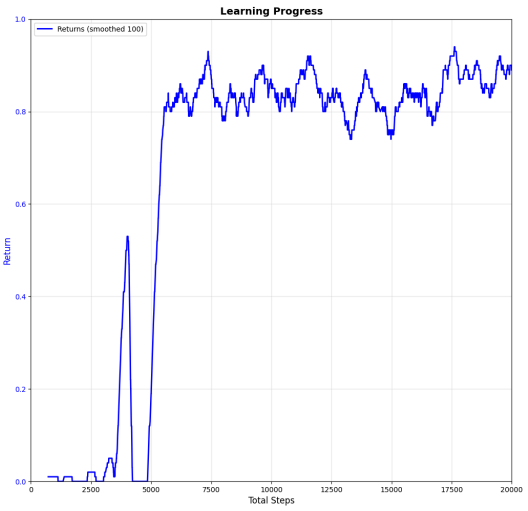
```
            # Compute current Q(S,A)
            q_values = Q_net(state_onehot)
            q_value = q_values[0, action]

            # Compute TD target: R + γ max_a Q(S',a)
            with torch.no_grad():
                next_q_values = Q_net(next_state_onehot)
                max_next_q = torch.max(next_q_values)
                td_target = reward + gamma * max_next_q

            # Compute loss (squared TD error)
            loss = criterion(q_value, td_target)

            # Update network (gradient descent step)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            episode_losses.append(loss.item())

            # Step 8: S ← S'
            state = next_state

            if truncated:
                done = True

        # Track statistics
        stats['episode_returns'].append(episode_reward)
        stats['episode_lengths'].append(episode_steps)
        stats['losses'].append(np.mean(episode_losses))
        stats['cumulative_steps'].append(total_steps)
        episode += 1

        # Print progress when rendering
        if use_render:
            recent_episodes = min(10, len(stats['episode_return
s']))
            avg_return = np.mean(stats['episode_returns'][-recent
_episodes:])
            print(f"Steps: {total_steps}/{max_total_steps}, Episo
des: {episode}, Avg Return (last {recent_episodes}): {avg_retur
n:.1f}")

        # Show progress with combined plot
        if show_animation and ((total_steps % plot_interval) == 0
or total_steps >= max_total_steps):
            Q_current = get_q_values_for_visualization(Q_net, num
_states, num_actions)
            if total_steps % (plot_interval * 2) < max_episode_st
eps:
                print(f"Steps: {total_steps}/{max_total_steps}")
            plot_q_values_and_learning_curves(Q_current, stats, t
otal_steps, max_total_steps)
            if total_steps < max_total_steps:
                time.sleep(0.3)

    # Extract greedy policy from Q-network
```

```
    Q_final = get_q_values_for_visualization(Q_net, num_states, n
um_actions)
    policy = np.argmax(Q_final, axis=1)

    # Close environments
    env.close()
    if env_render is not None:
        env_render.close()

    return Q_net, policy, stats
```

In [2]:

```
# Execute Q-learning with neural network on FrozenLake
import importlib
importlib.reload(helper_functions)
from helper_functions import plot_continuous_learning_curve

# Train the agent (with periodic Q-value visualization)
q_network, learned_policy, training_stats = q_learning_neural_net
work(
    env_name='FrozenLake-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon=0.1,
    hidden_sizes=[64, 64],
    show_animation=True,
    plot_interval=100
)
```



## Continuous State Spaces with Neural Networks

**Key Advantage:** Neural networks enable us to handle **continuous state spaces** that would be impossible with tabular methods.

**Examples of continuous state representations:**

- **Vision/Pixel-based input**: Raw images from Atari games, robotic cameras
- **Physical measurements**: Position, velocity, acceleration in continuous ranges
- **High-dimensional sensors**: LiDAR, depth sensors, joint angles

**Why this matters:**

- Tabular Q-learning required **discrete, finite state space** (e.g., 16 states in FrozenLake)
- Real-world problems have **infinite or very large state spaces**
- Neural networks can **generalize** across similar states through learned features

**Note on action spaces:**

- Q-learning (including neural network version) still requires **discrete action space**
- For continuous actions, we need policy gradient methods (coming in later lectures)

Let's demonstrate with CartPole, which has a **continuous 4-dimensional state space**:

- Cart position: $[-4.8, 4.8]$
- Cart velocity: $[-\infty, \infty]$
- Pole angle: $[-0.418, 0.418]$ rad
- Pole angular velocity: $[-\infty, \infty]$

In [3]:

```
# Q-Learning with Neural Network on CartPole (Continuous State Sp
ace)
import importlib
importlib.reload(helper_functions)
from helper_functions import plot_continuous_learning_curve

# Q-Network class for continuous state spaces (MLP architecture)
class QNetworkContinuous(nn.Module):
    """Q-Network for continuous state spaces using fully-connecte
d layers"""
    def __init__(self, n_state_dims, n_actions, hidden_sizes=[64,
64]):
        super(QNetworkContinuous, self).__init__()
        layers = []
        input_size = n_state_dims
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(input_size, hidden_size))
            layers.append(nn.ReLU())
            input_size = hidden_size
        layers.append(nn.Linear(input_size, n_actions))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

In [4]:
```
# Modular Q-Learning function that accepts any Q-network architec
ture
def q_learning_with_network(Q_net,
                            env_name='CartPole-v1',
                            max_total_steps=50000,
                            alpha=0.001,
                            gamma=0.99,
                            epsilon=0.1,
                            max_episode_steps=500,
                            render_interval=None,
                            preprocess_fn=None,
                            use_visual_obs=False):
    """
    Q-Learning with any Q-network architecture (modular design)

    Args:
        Q_net: Q-network instance (can be MLP, CNN, etc.)
        env_name: Environment name
        max_total_steps: Maximum total steps across all episodes
        alpha: Learning rate
        gamma: Discount factor
        epsilon: Exploration rate
        max_episode_steps: Maximum steps per episode
        render_interval: Render every N steps (None = no renderin
g)
        preprocess_fn: Optional function to preprocess observatio
ns
        use_visual_obs: If True, use env.render() for observation
s instead of state
    """
    # Training environment
    # Use rgb_array if using visual observations
    if use_visual_obs:
        env = gym.make(env_name, render_mode='rgb_array')
    else:
        env = gym.make(env_name)

    # Rendering environment (separate instance)
    env_render = None
    if render_interval is not None:
        if use_visual_obs:
            # Need rgb_array for visual observations even when re
ndering
            env_render = gym.make(env_name, render_mode='rgb_arra
y')
        else:
            env_render = gym.make(env_name, render_mode='human')

    # Get action space size
    n_actions = env.action_space.n

    print(f"Environment: {env_name}")
    print(f"Observation space: {env.observation_space}")
    print(f"Action space: {n_actions} discrete actions")
    print(f"Max total steps: {max_total_steps}")
    print(f"Max episode steps: {max_episode_steps}")
```

```
    if render_interval:
        print(f"Rendering: Every {render_interval} steps\n")
    else:
        print("Rendering: Disabled\n")

    # Set random seeds
    np.random.seed(0)
    torch.manual_seed(0)

    # Initialize optimizer and loss
    optimizer = optim.Adam(Q_net.parameters(), lr=alpha)
    criterion = nn.MSELoss()

    stats = {'episode_returns': [], 'episode_lengths': [], 'cumul
ative_steps': []}

    total_steps = 0
    episode = 0

    while total_steps < max_total_steps:
        # Use render environment at intervals
        use_render = (render_interval is not None and
                      total_steps % render_interval == 0)
        current_env = env_render if use_render else env

        # Reset and get initial observation
        if use_visual_obs:
            current_env.reset()
            state = current_env.render()
        else:
            state, _ = current_env.reset()
        if preprocess_fn is not None:
            state = preprocess_fn(state)

        episode_reward = 0
        episode_steps = 0
        done = False

        while not done and episode_steps < max_episode_steps and
total_steps < max_total_steps:
            # Epsilon-greedy action selection
            if np.random.rand() < epsilon:
                action = np.random.choice(n_actions)
            else:
                state_tensor = torch.FloatTensor(state).unsqueeze
(0)
                with torch.no_grad():
                    action = torch.argmax(Q_net(state_tensor)).it
em()

            # Take action and get reward
            next_state_raw, reward, done, truncated, info = curre
nt_env.step(action)

            # Get next observation - either visual or state
            if use_visual_obs:
                next_state = current_env.render()
```

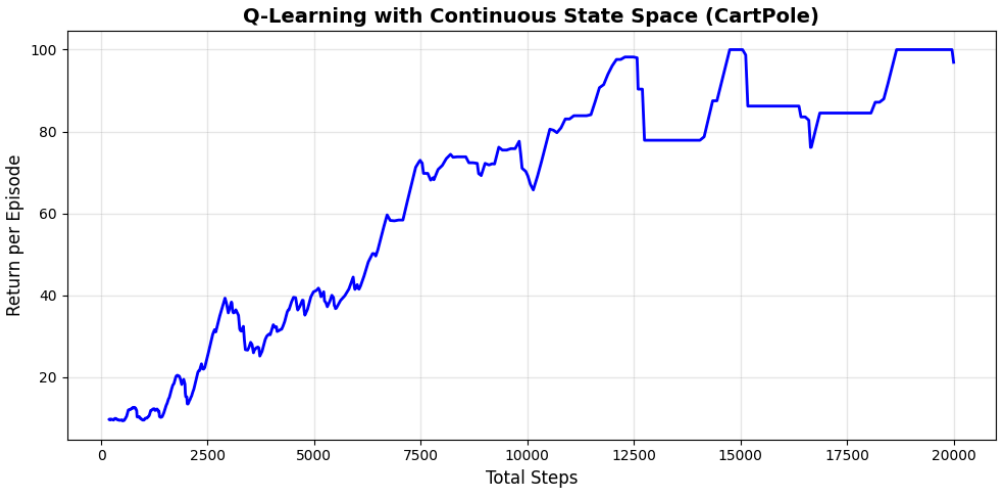
```
            else:
                next_state = next_state_raw
            if preprocess_fn is not None:
                next_state = preprocess_fn(next_state)

            episode_reward += reward
            episode_steps += 1
            total_steps += 1

            # Check if episode should end
            if truncated:
                done = True

            # Q-learning update
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            next_state_tensor = torch.FloatTensor(next_state).uns
queeze(0)

            # Current Q-value (requires grad) - use gather to mai
ntain gradient
            q_values = Q_net(state_tensor)
            action_tensor = torch.LongTensor([[action]])
            q_value = q_values.gather(1, action_tensor)  # Shape:
(1, 1)

            # Target Q-value (no grad)
            with torch.no_grad():
                next_q_values = Q_net(next_state_tensor)
                max_next_q = torch.max(next_q_values).item()
                # TD target: reward + gamma * max Q(s',a') if not
terminal
                td_target_val = reward + gamma * max_next_q * (1.
0 - float(done))
                td_target = torch.FloatTensor([[td_target_val]])
# Shape: (1, 1)

            # Compute loss and update
            loss = criterion(q_value, td_target)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            state = next_state
            # Print progress
            if render_interval is not None and (total_steps % ren
der_interval == 0):
                recent_episodes = min(10, len(stats['episode_retu
rns']))
                avg_return = np.mean(stats['episode_returns'][-re
cent_episodes:])
                print(f"Steps: {total_steps}/{max_total_steps}, E
pisodes: {episode}, Avg Return (last {recent_episodes}): {avg_ret
urn:.1f}")

        # Record statistics
        stats['episode_returns'].append(episode_reward)
        stats['episode_lengths'].append(episode_steps)
```

```
        stats['cumulative_steps'].append(total_steps)
        episode += 1


    env.close()
    if env_render is not None:
        env_render.close()

    return Q_net, stats
```

In [5]:

```
# Execute: Train on CartPole with continuous state space (vector
observations)
print("Training Q-learning with MLP network on continuous state s
pace...")
print("=" * 60)

# Create MLP Q-network for 4D continuous state
q_net_continuous = QNetworkContinuous(n_state_dims=4, n_actions=
2, hidden_sizes=[64, 64])

# Train with the modular Q-learning function
q_net_continuous, stats_continuous = q_learning_with_network(
    Q_net=q_net_continuous,
    env_name='CartPole-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon=0.1,
    max_episode_steps=100,
    render_interval=1000
)

# Plot results using helper function
plot_continuous_learning_curve(
    stats_continuous,
    window=20,
    title='Q-Learning with Continuous State Space (CartPole)'
)

print(f"\nFinal average return (last 50 episodes): {np.mean(stats
_continuous['episode_returns'][-50:]):.1f}")
```

```
Training Q-learning with MLP network on continuous state space...
============================================================
Environment: CartPole-v1
Observation space: Box([-4.8               -inf -0.41887903
-inf], [4.8               inf 0.41887903        inf], (4,), float
32)
Action space: 2 discrete actions
Max total steps: 20000
Max episode steps: 100
Rendering: Every 1000 steps

Steps: 1000/20000, Episodes: 98, Avg Return (last 10): 9.1
Steps: 2000/20000, Episodes: 166, Avg Return (last 10): 16.1
Steps: 3000/20000, Episodes: 198, Avg Return (last 10): 36.6
Steps: 4000/20000, Episodes: 233, Avg Return (last 10): 31.9
Steps: 5000/20000, Episodes: 258, Avg Return (last 10): 42.0
Steps: 6000/20000, Episodes: 282, Avg Return (last 10): 48.5
Steps: 7000/20000, Episodes: 298, Avg Return (last 10): 71.2
Steps: 8000/20000, Episodes: 312, Avg Return (last 10): 52.4
Steps: 9000/20000, Episodes: 326, Avg Return (last 10): 65.9
Steps: 10000/20000, Episodes: 339, Avg Return (last 10): 78.9
Steps: 11000/20000, Episodes: 351, Avg Return (last 10): 92.1
Steps: 12000/20000, Episodes: 361, Avg Return (last 10): 100.0
Steps: 13000/20000, Episodes: 375, Avg Return (last 10): 55.7
Steps: 14000/20000, Episodes: 385, Avg Return (last 10): 100.0
Steps: 15000/20000, Episodes: 395, Avg Return (last 10): 100.0
Steps: 16000/20000, Episodes: 408, Avg Return (last 10): 83.3
Steps: 17000/20000, Episodes: 421, Avg Return (last 10): 69.0
Steps: 18000/20000, Episodes: 431, Avg Return (last 10): 100.0
Steps: 19000/20000, Episodes: 441, Avg Return (last 10): 100.0
Steps: 20000/20000, Episodes: 451, Avg Return (last 10): 100.0
```




```
Final average return (last 50 episodes): 92.6
```

## Extending Neural Q-Learning to Visual Observations

**From vectors to images**: The same Q-learning algorithm can handle visual inputs by replacing the fully-connected network with a **Convolutional Neural Network (CNN)**:

**CNN Architecture for Q-Learning:**

- **Convolutional layers**: Extract spatial features from pixel observations (edges, shapes, patterns)
- **Flatten layer**: Convert 2D feature maps to 1D vector3. Pass the appropriate network based on observation type
- **Fully-connected layers**: Map features to Q-values for each action2. Define different network architectures (MLP for vectors, CNN for images)

**Key insight**: The Q-learning algorithm remains identical - only the network architecture changes to match the input type.**Implementation strategy:**

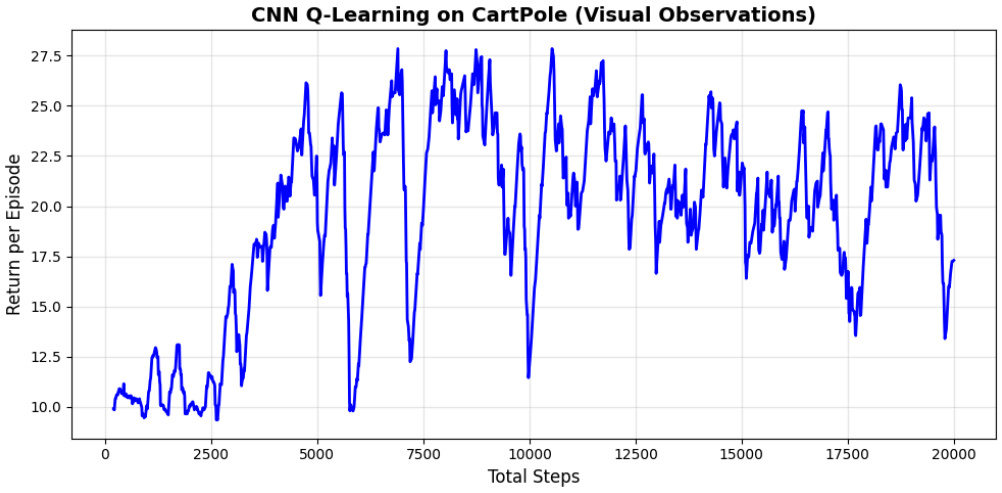
In [6]:

```
# CNN-based Q-Network for visual observations (lightweight versio
n)
class QNetworkCNN(nn.Module):
    """Q-Network with CNN architecture for processing image obser
vations"""
    def __init__(self, n_actions, input_channels=3, img_height=4
2, img_width=42):
        super(QNetworkCNN, self).__init__()

        # Convolutional layers for feature extraction (reduced si
ze)
        self.conv = nn.Sequential(
            nn.Conv2d(input_channels, 16, kernel_size=4, stride=
2),  # Reduced from 32 channels
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=2),  # Reduce
d from 64 channels
            nn.ReLU()
        )

        # Calculate size after conv layers
        def conv_output_size(size, kernel_size, stride):
            return (size - kernel_size) // stride + 1

        conv_h = conv_output_size(conv_output_size(img_height, 4,
2), 3, 2)
        conv_w = conv_output_size(conv_output_size(img_width, 4,
2), 3, 2)
        conv_output_size_total = conv_h * conv_w * 32

        # Fully connected layers (reduced size)
        self.fc = nn.Sequential(
            nn.Linear(conv_output_size_total, 128),  # Reduced fr
om 512
            nn.ReLU(),
            nn.Linear(128, n_actions)
        )

    def forward(self, x):
        # x shape: (batch, channels, height, width)
        x = self.conv(x)
        x = x.view(x.size(0), -1)  # Flatten
        return self.fc(x)
```

In [7]:

```
print("=" * 60)
print("🎮 Q-Learning with CNN on CartPole (Visual Observations)")
print("=" * 60)

from helper_functions import preprocess_image

# Test preprocessing
env_test = gym.make('CartPole-v1', render_mode='rgb_array')
env_test.reset()
sample_img = env_test.render()
print(f"Visual observation shape: {sample_img.shape}")
processed = preprocess_image(sample_img)
print(f"Preprocessed shape: {processed.shape}")
env_test.close()

# Create CNN Q-network for visual observations (42x42 grayscale i
nput)
q_net_cnn = QNetworkCNN(
    n_actions=2,
    input_channels=1,  # Grayscale
    img_height=42,
    img_width=42
)

# Train with visual observations
print("\nStarting training with visual observations...")
q_net_cnn, q_net_stats = q_learning_with_network(
    Q_net=q_net_cnn,
    env_name='CartPole-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon=0.1,
    max_episode_steps=100,
    use_visual_obs=True,
    preprocess_fn=preprocess_image,
    render_interval=1000
)

# Plot results
plot_continuous_learning_curve(q_net_stats, window=20,
                               title="CNN Q-Learning on CartPole
(Visual Observations)")
```

```
============================================================
🎮 Q-Learning with CNN on CartPole (Visual Observations)
============================================================
Visual observation shape: (400, 600, 3)
Preprocessed shape: (1, 42, 42)

Starting training with visual observations...
Environment: CartPole-v1
Observation space: Box([-4.8               -inf -0.41887903
-inf], [4.8               inf 0.41887903        inf], (4,), float
32)
Action space: 2 discrete actions
Max total steps: 20000
Max episode steps: 100
Rendering: Every 1000 steps

Steps: 1000/20000, Episodes: 97, Avg Return (last 10): 10.3
Steps: 2000/20000, Episodes: 187, Avg Return (last 10): 9.9
Steps: 3000/20000, Episodes: 271, Avg Return (last 10): 14.6
Steps: 4000/20000, Episodes: 334, Avg Return (last 10): 16.3
Steps: 5000/20000, Episodes: 378, Avg Return (last 10): 18.6
Steps: 6000/20000, Episodes: 439, Avg Return (last 10): 14.4
Steps: 7000/20000, Episodes: 479, Avg Return (last 10): 24.1
Steps: 8000/20000, Episodes: 527, Avg Return (last 10): 25.6
Steps: 9000/20000, Episodes: 568, Avg Return (last 10): 21.2
Steps: 10000/20000, Episodes: 621, Avg Return (last 10): 11.2
Steps: 11000/20000, Episodes: 663, Avg Return (last 10): 18.7
Steps: 12000/20000, Episodes: 704, Avg Return (last 10): 25.6
Steps: 13000/20000, Episodes: 754, Avg Return (last 10): 17.5
Steps: 14000/20000, Episodes: 803, Avg Return (last 10): 20.0
Steps: 15000/20000, Episodes: 846, Avg Return (last 10): 15.4
Steps: 16000/20000, Episodes: 900, Avg Return (last 10): 16.7
Steps: 17000/20000, Episodes: 944, Avg Return (last 10): 24.5
Steps: 18000/20000, Episodes: 1002, Avg Return (last 10): 23.1
Steps: 19000/20000, Episodes: 1044, Avg Return (last 10): 21.3
Steps: 20000/20000, Episodes: 1095, Avg Return (last 10): 20.7
```



# Towards Deep Q-Network (DQN)

While neural network Q-learning works, it suffers from instability issues that make training unreliable:

**Problems with naive neural network Q-learning:**

- **Correlated samples**: Sequential experiences are highly correlated (similar states, temporal dependencies), leading to inefficient learning and poor convergence
- **Non-stationary targets**: The TD target $r + \gamma \max_{a'} Q_\theta(s', a')$ constantly changes as the Q-network updates, causing training instability

**DQN improvements (Mnih et al., 2015):**

1. **Experience Replay**: Store experiences $(s, a, r, s')$ in a replay buffer and sample random mini-batches for training
    - Breaks correlation between consecutive samples (decorrelation)
    - Reduces gradient variance by averaging over mini-batch samples
    - Enables reusing experiences multiple times (better sample efficiency)
2. **Target Network**: Use a separate, slowly-updated network $Q_{\theta^-}$ for computing TD targets: $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$
    - Stabilizes training by keeping targets fixed for multiple updates
    - Updated periodically (every C steps) by copying weights: $\theta^- \leftarrow \theta$
3. **Frame Stacking**: Stack last N frames as input to capture temporal information and motion (originally N=4)
4. **Frame Skipping**: Repeat each selected action for N frames to reduce computational cost (originally N=4)
5. **Reward Clipping**: Clip rewards to $[-1, +1]$ for stability across different games
6. **Epsilon Decay**: Linearly anneal exploration rate $\epsilon$ from 1.0 to 0.1 over training

## Experience Replay: Breaking Correlations

### Experience Replay: Data Reuse and Decorrelation

Store transitions in buffer, sample random mini-batches for training to break temporal correlations.

([Levine et al., CS285 Lecture 8, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf))

**Replay Buffer:**

- Stores: $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$
- Fixed size capacity, e.g. $N = 10^5$ to $N = 10^6$ transitions
- Policy: FIFO (oldest replaced)

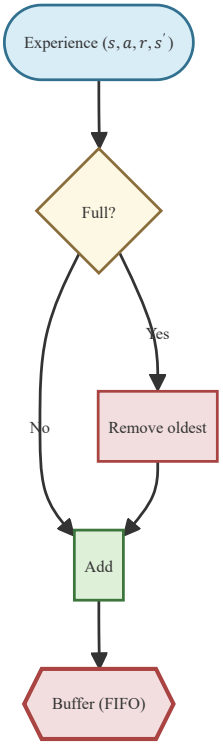
**Training:**

1. Store new experience: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$
2. Sample random mini-batch: $B \sim \mathcal{D}$
3. Compute loss on $B$
4. Update $\theta$

**Benefits:**

- **Decorrelation**: Breaks temporal correlations in data
- **Variance reduction**: Batch learning averages gradients over multiple samples, reducing gradient variance
- **Sample efficiency**: Enables data reuse (each experience used multiple times)
- **Stability**: Approximates IID assumption for SGD convergence

## Implementing Replay Buffers




**Replay Buffer Structure**

Finite-sized circular buffer (FIFO queue) storing experience tuples.

([Eysenbach, COS435 Lecture 11: Deep Q Learning, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf))

**Data Structure:**

- Tuple: $(s, a, r, s', \text{done})$
- Buffer size: Typically 10k - 1M transitions
- Policy: Oldest experiences discarded when full (FIFO)

In [8]:
```
print("=" * 60)
print("🎮 Q-Learning with Experience Replay Buffer")
print("=" * 60)

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym
from collections import deque
import random

class ReplayBuffer:
    """Experience Replay Buffer for storing and sampling transiti
ons"""
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """Store a transition in the buffer"""
        self.buffer.append((state, action, reward, next_state, do
ne))

    def sample(self, batch_size):
        """Sample a random batch of transitions"""
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batc
h)
        return (np.array(states), np.array(actions), np.array(rew
ards),
                np.array(next_states), np.array(dones))

    def __len__(self):
        return len(self.buffer)

def q_learning_with_replay(Q_net,
                           env_name='CartPole-v1',
                           max_total_steps=50000,
                           alpha=0.001,
                           gamma=0.99,
                           epsilon=0.1,
                           max_episode_steps=500,
                           batch_size=32,
                           buffer_capacity=10000,
                           min_buffer_size=1000,
                           render_interval=None,
                           preprocess_fn=None,
                           use_visual_obs=False):
    """
    Q-Learning with Experience Replay Buffer

    Args:
        Q_net: Q-network instance
        env_name: Environment name
        max_total_steps: Maximum total steps across all episodes
        alpha: Learning rate
```

```
        gamma: Discount factor
        epsilon: Exploration rate (fixed)
        max_episode_steps: Maximum steps per episode
        batch_size: Size of mini-batch sampled from replay buffer
        buffer_capacity: Maximum size of replay buffer
        min_buffer_size: Minimum buffer size before training star
ts
        render_interval: Render every N steps (None = no renderin
g)
        preprocess_fn: Optional function to preprocess observatio
ns
        use_visual_obs: If True, use env.render() for observation
s
    """
    # Create environment
    if use_visual_obs:
        env = gym.make(env_name, render_mode='rgb_array')
    else:
        env = gym.make(env_name)

    # Rendering environment (separate instance)
    env_render = None
    if render_interval is not None:
        if use_visual_obs:
            env_render = gym.make(env_name, render_mode='rgb_arra
y')
        else:
            env_render = gym.make(env_name, render_mode='human')

    n_actions = env.action_space.n

    print(f"Environment: {env_name}")
    print(f"Action space: {n_actions} discrete actions")
    print(f"Replay buffer capacity: {buffer_capacity}")
    print(f"Batch size: {batch_size}")
    print(f"Max total steps: {max_total_steps}")
    print(f"Max episode steps: {max_episode_steps}")
    if render_interval:
        print(f"Rendering: Every {render_interval} steps\n")
    else:
        print("Rendering: Disabled\n")

    # Set random seeds
    np.random.seed(0)
    torch.manual_seed(0)
    random.seed(0)

    # Initialize replay buffer
    replay_buffer = ReplayBuffer(buffer_capacity)

    # Initialize optimizer and loss
    optimizer = optim.Adam(Q_net.parameters(), lr=alpha)
    criterion = nn.MSELoss()

    # Statistics
    stats = {'episode_returns': [], 'episode_lengths': [], 'cumul
ative_steps': []}
```

```
    total_steps = 0
    episode = 0

    while total_steps < max_total_steps:
        # Use render environment at intervals
        use_render = (render_interval is not None and
                      total_steps % render_interval < max_episode_
steps)
        current_env = env_render if use_render else env

        # Reset environment
        current_env.reset()
        if use_visual_obs:
            state = current_env.render()
        else:
            state, _ = current_env.reset()
        if preprocess_fn is not None:
            state = preprocess_fn(state)

        episode_reward = 0
        episode_steps = 0
        done = False

        while not done and episode_steps < max_episode_steps and
total_steps < max_total_steps:
            # Epsilon-greedy action selection
            if np.random.rand() < epsilon:
                action = np.random.choice(n_actions)
            else:
                state_tensor = torch.FloatTensor(state).unsqueeze
(0)
                with torch.no_grad():
                    action = torch.argmax(Q_net(state_tensor)).it
em()

            # Take action
            next_state_raw, reward, done, truncated, info = curre
nt_env.step(action)

            # Get next observation
            if use_visual_obs:
                next_state = current_env.render()
            else:
                next_state = next_state_raw
            if preprocess_fn is not None:
                next_state = preprocess_fn(next_state)

            episode_reward += reward
            episode_steps += 1
            total_steps += 1

            if truncated:
                done = True

            # Store transition in replay buffer
            replay_buffer.push(state, action, reward, next_state,
```

```
done)

            # Train if buffer has enough samples
            if len(replay_buffer) >= min_buffer_size:
                # Sample mini-batch from replay buffer
                states, actions, rewards, next_states, dones = re
play_buffer.sample(batch_size)

                # Convert to tensors
                states_tensor = torch.FloatTensor(states)
                actions_tensor = torch.LongTensor(actions).unsque
eze(1)
                rewards_tensor = torch.FloatTensor(rewards).unsqu
eeze(1)
                next_states_tensor = torch.FloatTensor(next_state
s)
                dones_tensor = torch.FloatTensor(dones).unsqueeze
(1)

                # Current Q-values
                q_values = Q_net(states_tensor).gather(1, actions
_tensor)

                # Target Q-values
                with torch.no_grad():
                    next_q_values = Q_net(next_states_tensor)
                    max_next_q_values = next_q_values.max(1, keep
dim=True)[0]
                    targets = rewards_tensor + gamma * max_next_q
_values * (1 - dones_tensor)

                # Compute loss and update
                loss = criterion(q_values, targets)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

            state = next_state
            # Print progress
            if render_interval is not None and total_steps % rend
er_interval == 0:
                recent_episodes = min(10, len(stats['episode_retu
rns']))
                avg_return = np.mean(stats['episode_returns'][-re
cent_episodes:])
                print(f"Steps: {total_steps}/{max_total_steps}, E
pisodes: {episode}, Avg Return (last {recent_episodes}): {avg_ret
urn:.1f}")

        # Record statistics
        stats['episode_returns'].append(episode_reward)
        stats['episode_lengths'].append(episode_steps)
        stats['cumulative_steps'].append(total_steps)
        episode += 1
```

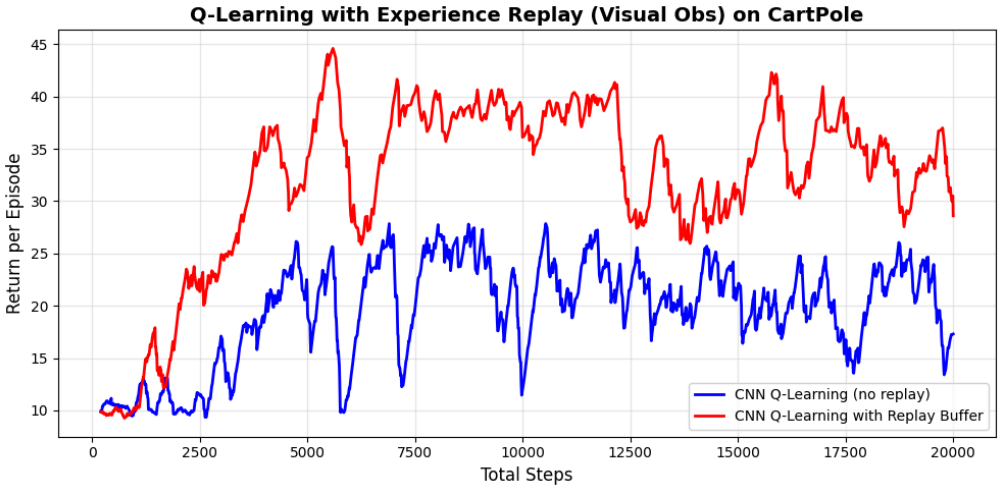
```
    env.close()
    if env_render is not None:
        env_render.close()

    return Q_net, stats
```

```
============================================================
🎮 Q-Learning with Experience Replay Buffer
============================================================
```

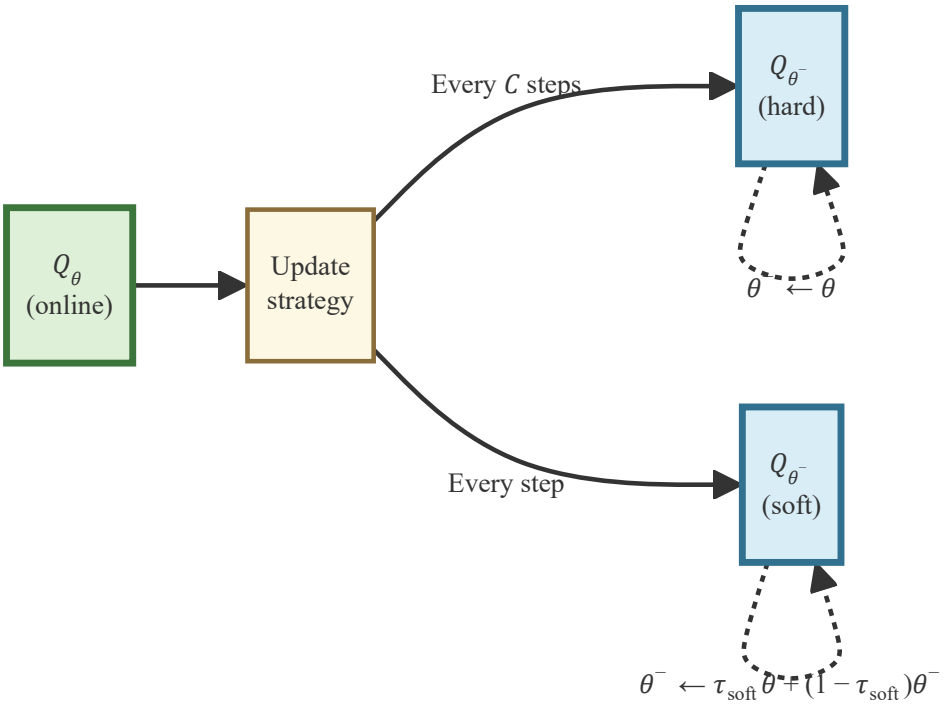
In [9]:

```
# Train Q-network with experience replay on CartPole with VISUAL
observations
from helper_functions import preprocess_image

q_net_replay = QNetworkCNN(n_actions=2, input_channels=1)

q_net_replay, q_net_replay_buffer_stats = q_learning_with_replay(
    Q_net=q_net_replay,
    env_name='CartPole-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon=0.1,
    batch_size=32,
    buffer_capacity=10000,
    max_episode_steps=500,
    min_buffer_size=1000,
    render_interval=1000,
    preprocess_fn=preprocess_image,
    use_visual_obs=True
)

# Plot results
plot_continuous_learning_curve(
    [q_net_stats, q_net_replay_buffer_stats],
    window=20,
    labels=['CNN Q-Learning (no replay)', 'CNN Q-Learning with Re
play Buffer'],
    title="Q-Learning with Experience Replay (Visual Obs) on Cart
Pole"
)
```

```
Environment: CartPole-v1
Action space: 2 discrete actions
Replay buffer capacity: 10000
Batch size: 32
Max total steps: 20000
Max episode steps: 500
Rendering: Every 1000 steps

Steps: 1000/20000, Episodes: 101, Avg Return (last 10): 10.1
Steps: 1000/20000, Episodes: 101, Avg Return (last 10): 10.1
Steps: 2000/20000, Episodes: 164, Avg Return (last 10): 22.9
Steps: 2000/20000, Episodes: 164, Avg Return (last 10): 22.9
Steps: 3000/20000, Episodes: 208, Avg Return (last 10): 25.1
Steps: 3000/20000, Episodes: 208, Avg Return (last 10): 25.1
Steps: 4000/20000, Episodes: 239, Avg Return (last 10): 35.6
Steps: 4000/20000, Episodes: 239, Avg Return (last 10): 35.6
Steps: 5000/20000, Episodes: 268, Avg Return (last 10): 42.8
Steps: 5000/20000, Episodes: 268, Avg Return (last 10): 42.8
Steps: 6000/20000, Episodes: 297, Avg Return (last 10): 25.7
Steps: 6000/20000, Episodes: 297, Avg Return (last 10): 25.7
Steps: 7000/20000, Episodes: 325, Avg Return (last 10): 40.8
Steps: 7000/20000, Episodes: 325, Avg Return (last 10): 40.8
Steps: 8000/20000, Episodes: 351, Avg Return (last 10): 39.1
Steps: 8000/20000, Episodes: 351, Avg Return (last 10): 39.1
Steps: 9000/20000, Episodes: 378, Avg Return (last 10): 36.5
Steps: 9000/20000, Episodes: 378, Avg Return (last 10): 36.5
Steps: 10000/20000, Episodes: 404, Avg Return (last 10): 36.2
Steps: 10000/20000, Episodes: 404, Avg Return (last 10): 36.2
Steps: 11000/20000, Episodes: 430, Avg Return (last 10): 35.9
Steps: 11000/20000, Episodes: 430, Avg Return (last 10): 35.9
Steps: 12000/20000, Episodes: 455, Avg Return (last 10): 37.9
Steps: 12000/20000, Episodes: 455, Avg Return (last 10): 37.9
Steps: 13000/20000, Episodes: 488, Avg Return (last 10): 29.1
Steps: 13000/20000, Episodes: 488, Avg Return (last 10): 29.1
Steps: 14000/20000, Episodes: 521, Avg Return (last 10): 30.3
Steps: 14000/20000, Episodes: 521, Avg Return (last 10): 30.3
Steps: 15000/20000, Episodes: 555, Avg Return (last 10): 31.1
Steps: 15000/20000, Episodes: 555, Avg Return (last 10): 31.1
Steps: 16000/20000, Episodes: 580, Avg Return (last 10): 36.7
Steps: 16000/20000, Episodes: 580, Avg Return (last 10): 36.7
Steps: 17000/20000, Episodes: 609, Avg Return (last 10): 38.1
Steps: 17000/20000, Episodes: 609, Avg Return (last 10): 38.1
Steps: 18000/20000, Episodes: 638, Avg Return (last 10): 35.1
Steps: 18000/20000, Episodes: 638, Avg Return (last 10): 35.1
Steps: 19000/20000, Episodes: 669, Avg Return (last 10): 29.0
Steps: 19000/20000, Episodes: 669, Avg Return (last 10): 29.0
Steps: 20000/20000, Episodes: 701, Avg Return (last 10): 21.3
Steps: 20000/20000, Episodes: 701, Avg Return (last 10): 21.3
```


Q-Learning with Experience Replay (Visual Obs) on CartPole
Return per Episode
Total Steps
CNN Q-Learning (no replay)
CNN Q-Learning with Replay Buffer
45
40
35
30
25
20
15
10
0
2500
5000
7500
10000
12500
15000
17500
20000

## Target Network Update Strategies




### Target Network Update Strategies

Two approaches for updating target network parameters from online network.

([Levine et al., CS285 Lecture 8, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf))

**Hard Update (Original DQN):**

$$\theta^- \leftarrow \theta \quad \text{every } C \text{ steps}$$

- Complete replacement at intervals
- Typical: $C = 1000$ - $10000$ steps
- Simple but can cause sudden changes

**Soft Update (DDPG, TD3):**

$$\theta^- \leftarrow \tau_{\text{soft}}\theta + (1 - \tau_{\text{soft}})\theta^-$$

- Gradual blending every step
- Typical: $\tau_{\text{soft}} = 0.001$ - $0.01$
- Smoother updates, more stable

**Comparison:**

| Method | Update Frequency | Smoothness | Use Case |
|---|---|---|---|
| Hard | Every $C$ steps | Discrete jumps | DQN, stable environments |

| Method | Update Frequency | Smoothness | Use Case |
|---|---|---|---|
| Soft | Every step | Smooth | DDPG, continuous control |

In [10]:
```
print("=" * 60)
print("🎮 DQN with Visual Observations (Replay + Target Networ
k)")
print("=" * 60)

def q_learning_with_replay_and_target(Q_net,
                                      env_name='CartPole-v1',
                                      max_total_steps=20000,
                                      alpha=0.001,
                                      gamma=0.99,
                                      epsilon=0.1,
                                      max_episode_steps=100,
                                      batch_size=32,
                                      buffer_capacity=10000,
                                      min_buffer_size=1000,
                                      target_update_freq=100,
                                      render_interval=None,
                                      preprocess_fn=None,
                                      use_visual_obs=False):
    """
    Q-Learning with Experience Replay Buffer AND Target Network
(Hard Updates)

    Args:
        Q_net: Q-network instance (will be cloned for target netw
ork)
        env_name: Environment name
        max_total_steps: Maximum total steps across all episodes
        alpha: Learning rate
        gamma: Discount factor
        epsilon: Exploration rate (fixed)
        max_episode_steps: Maximum steps per episode
        batch_size: Size of mini-batch sampled from replay buffer
        buffer_capacity: Maximum size of replay buffer
        min_buffer_size: Minimum buffer size before training star
ts
        target_update_freq: Update target network every N steps
(hard update)
        render_interval: Render every N steps (None = no renderin
g)
        preprocess_fn: Optional function to preprocess observatio
ns
        use_visual_obs: If True, use env.render() for observation
s
    """
    import copy

    # Create environment
    if use_visual_obs:
        env = gym.make(env_name, render_mode='rgb_array')
    else:
        env = gym.make(env_name)

    # Rendering environment (separate instance)
    env_render = None
    if render_interval is not None:
```

```
        if use_visual_obs:
            env_render = gym.make(env_name, render_mode='rgb_arra
y')
        else:
            env_render = gym.make(env_name, render_mode='human')

    n_actions = env.action_space.n

    print(f"Environment: {env_name}")
    print(f"Action space: {n_actions} discrete actions")
    print(f"Replay buffer capacity: {buffer_capacity}")
    print(f"Batch size: {batch_size}")
    print(f"Target network update frequency: {target_update_freq}
steps")
    print(f"Max total steps: {max_total_steps}")
    print(f"Max episode steps: {max_episode_steps}")
    if render_interval:
        print(f"Rendering: Every {render_interval} steps\n")
    else:
        print("Rendering: Disabled\n")

    # Set random seeds
    np.random.seed(0)
    torch.manual_seed(0)
    random.seed(0)

    # Initialize replay buffer
    replay_buffer = ReplayBuffer(buffer_capacity)

    # Create target network (clone of Q_net)
    Q_target = copy.deepcopy(Q_net)
    Q_target.load_state_dict(Q_net.state_dict())

    # Initialize optimizer and loss
    optimizer = optim.Adam(Q_net.parameters(), lr=alpha)
    criterion = nn.MSELoss()

    # Statistics
    stats = {'episode_returns': [], 'episode_lengths': [], 'cumul
ative_steps': []}

    total_steps = 0
    episode = 0

    while total_steps < max_total_steps:
        # Use render environment at intervals
        use_render = (render_interval is not None and
                      total_steps % render_interval < max_episode_
steps)
        current_env = env_render if use_render else env

        # Reset environment
        current_env.reset()
        if use_visual_obs:
            state = current_env.render()
        else:
            state, _ = current_env.reset()
```

```
        if preprocess_fn is not None:
            state = preprocess_fn(state)

        episode_reward = 0
        episode_steps = 0
        done = False

        while not done and episode_steps < max_episode_steps and
total_steps < max_total_steps:
            # Epsilon-greedy action selection (using online Q_ne
t)
            if np.random.rand() < epsilon:
                action = np.random.choice(n_actions)
            else:
                state_tensor = torch.FloatTensor(state).unsqueeze
(0)
                with torch.no_grad():
                    action = torch.argmax(Q_net(state_tensor)).it
em()

            # Take action
            next_state_raw, reward, done, truncated, info = curre
nt_env.step(action)

            # Get next observation
            if use_visual_obs:
                next_state = current_env.render()
            else:
                next_state = next_state_raw
            if preprocess_fn is not None:
                next_state = preprocess_fn(next_state)

            episode_reward += reward
            episode_steps += 1
            total_steps += 1

            if truncated:
                done = True

            # Store transition in replay buffer
            replay_buffer.push(state, action, reward, next_state,
done)

            # Train if buffer has enough samples
            if len(replay_buffer) >= min_buffer_size:
                # Sample mini-batch from replay buffer
                states, actions, rewards, next_states, dones = re
play_buffer.sample(batch_size)

                # Convert to tensors
                states_tensor = torch.FloatTensor(states)
                actions_tensor = torch.LongTensor(actions).unsque
eze(1)
                rewards_tensor = torch.FloatTensor(rewards).unsqu
eeze(1)
                next_states_tensor = torch.FloatTensor(next_state
s)
```

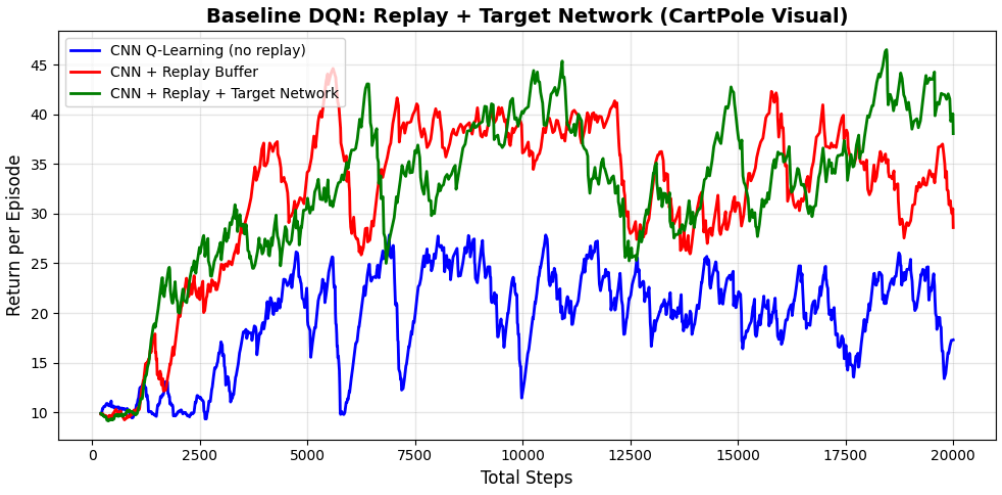
```
                dones_tensor = torch.FloatTensor(dones).unsqueeze
(1)

                # Current Q-values (from online network)
                q_values = Q_net(states_tensor).gather(1, actions
_tensor)

                # Target Q-values (using target network - KEY DIF
FERENCE!)
                with torch.no_grad():
                    next_q_values = Q_target(next_states_tensor)
                    max_next_q_values = next_q_values.max(1, keep
dim=True)[0]
                    targets = rewards_tensor + gamma * max_next_q
_values * (1 - dones_tensor)

                # Compute loss and update online network
                loss = criterion(q_values, targets)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                # Hard update target network every target_update_
freq steps
                if total_steps % target_update_freq == 0:
                    Q_target.load_state_dict(Q_net.state_dict())

            state = next_state
            # Print progress
            if render_interval is not None and (total_steps % ren
der_interval == 0):
                recent_episodes = min(10, len(stats['episode_retu
rns']))
                avg_return = np.mean(stats['episode_returns'][-re
cent_episodes:])
                print(f"Steps: {total_steps}/{max_total_steps}, E
pisodes: {episode}, Avg Return (last {recent_episodes}): {avg_ret
urn:.1f}")

        # Record statistics
        stats['episode_returns'].append(episode_reward)
        stats['episode_lengths'].append(episode_steps)
        stats['cumulative_steps'].append(total_steps)
        episode += 1



    env.close()
    if env_render is not None:
        env_render.close()

    return Q_net, stats
```

```
============================================================
🎮 DQN with Visual Observations (Replay + Target Network)
============================================================
```

In [11]:

```
# Train baseline DQN with Replay Buffer + Target Network (no othe
r enhancements)
from helper_functions import preprocess_image

print("=" * 60)
print("🔧 Q-Learning with Replay Buffer and Target Network")
print("=" * 60)

# Create CNN Q-network for visual observations (baseline)
q_net_replay_buffer_target_net = QNetworkCNN(
    n_actions=2,
    input_channels=1,
    img_height=42,
    img_width=42
)

q_net_replay_buffer_target_net, q_net_replay_buffer_target_net_st
ats = q_learning_with_replay_and_target(
    Q_net=q_net_replay_buffer_target_net,
    env_name='CartPole-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon=0.1,  # Fixed epsilon (no annealing)
    batch_size=32,
    buffer_capacity=10000,
    min_buffer_size=1000,
    target_update_freq=100,
    use_visual_obs=True,     # Use visual observations
    preprocess_fn=preprocess_image,  # Preprocess RGB to grayscal
e
    render_interval=1000,
    max_episode_steps=100
)

# Plot results
plot_continuous_learning_curve(
    [q_net_stats, q_net_replay_buffer_stats, q_net_replay_buffer_
target_net_stats],
    window=20,
    labels=['CNN Q-Learning (no replay)', 'CNN + Replay Buffer',
'CNN + Replay + Target Network'],
    title="Baseline DQN: Replay + Target Network (CartPole Visua
l)"
)

print(f"\\nBaseline final average return (last 50 episodes): {np.
mean(q_net_replay_buffer_target_net_stats['episode_returns'][-5
0:]):.1f}")
```

```
============================================================
🔧 Q-Learning with Replay Buffer and Target Network
============================================================
Environment: CartPole-v1
Action space: 2 discrete actions
Replay buffer capacity: 10000
Batch size: 32
Target network update frequency: 100 steps
Max total steps: 20000
Max episode steps: 100
Rendering: Every 1000 steps

Steps: 1000/20000, Episodes: 102, Avg Return (last 10): 9.7
Steps: 2000/20000, Episodes: 154, Avg Return (last 10): 19.2
Steps: 3000/20000, Episodes: 192, Avg Return (last 10): 29.9
Steps: 4000/20000, Episodes: 228, Avg Return (last 10): 27.9
Steps: 5000/20000, Episodes: 262, Avg Return (last 10): 30.4
Steps: 6000/20000, Episodes: 291, Avg Return (last 10): 43.3
Steps: 7000/20000, Episodes: 320, Avg Return (last 10): 30.9
Steps: 8000/20000, Episodes: 351, Avg Return (last 10): 32.7
Steps: 9000/20000, Episodes: 377, Avg Return (last 10): 39.1
Steps: 10000/20000, Episodes: 402, Avg Return (last 10): 46.2
Steps: 11000/20000, Episodes: 426, Avg Return (last 10): 44.1
Steps: 12000/20000, Episodes: 456, Avg Return (last 10): 31.9
Steps: 13000/20000, Episodes: 489, Avg Return (last 10): 37.1
Steps: 14000/20000, Episodes: 524, Avg Return (last 10): 29.1
Steps: 15000/20000, Episodes: 549, Avg Return (last 10): 38.4
Steps: 16000/20000, Episodes: 580, Avg Return (last 10): 35.0
Steps: 17000/20000, Episodes: 609, Avg Return (last 10): 38.9
Steps: 18000/20000, Episodes: 636, Avg Return (last 10): 41.9
Steps: 19000/20000, Episodes: 660, Avg Return (last 10): 42.8
Steps: 20000/20000, Episodes: 685, Avg Return (last 10): 41.6
```




```
\nBaseline final average return (last 50 episodes): 41.2
```

## Recap and Next Steps: Towards the "original" DQN as in "Human-level control through deep reinforcement learning" (Mnih et al., 2015, Nature)

- **Vanilla neural Q-Learning**: Works well for low-dimensional, tabular or small vector state spaces when paired with simple function approximators (or tables).
- **High-dimensional inputs (images)**: Require neural networks; training becomes unstable due to correlated samples and non-stationary targets.
- **Replay Buffer**: Replay buffers decorrelate samples and enable batch updates (lower gradient variance).
- **Target Network**: Target networks stabilize TD targets via slow/hard updates.

Now we reproduce further stabilizing additions from the 2015 DQN paper:

- **Frame Stacking**: Stack the last N frames to capture short-term dynamics and motion cues (Atari setting) (N=4 in DQN paper).
- **Frame Skipping**: Consider every N-th frame and repeat action N times (N=4 in DQN paper).
- **Reward Clipping**: Clip rewards to $[-1, +1]$ to normalize scales and improve stability across games.
- **Exploration Schedule (ε-greedy)**: $\epsilon$ linearly annealed. In the paper: from 1.0 → 0.1 over 1M frames.

In [12]:
```
print("=" * 60)
print("🎮 DQN with Visual Observations (Replay + Target Networ
k)")
print("=" * 60)

def dqn_2015(Q_net,
             env_name='CartPole-v1',
             max_total_steps=20000,
             alpha=0.001,
             gamma=0.99,
             epsilon_start=1.0,
             epsilon_end=0.1,
             epsilon_decay_steps=10000,
             max_episode_steps=100,
             batch_size=32,
             buffer_capacity=10000,
             min_buffer_size=1000,
             target_update_freq=100,
             frame_stack=4,
             frame_skip=4,
             clip_rewards=True,
             render_interval=None,
             preprocess_fn=None,
             use_visual_obs=False):
    """
    DQN 2015 with all features from "Human-level control through
deep RL" (Nature)

    Features:
    - Experience Replay Buffer
    - Target Network (Hard Updates)
    - Frame Stacking: Stack last N frames to capture motion
    - Frame Skipping: Repeat action for N frames
    - Reward Clipping: Clip to [-1, +1]
    - Epsilon Annealing: Linear decay from epsilon_start to epsil
on_end

    Args:
        Q_net: Q-network instance (will be cloned for target netw
ork)
        env_name: Environment name
        max_total_steps: Maximum total steps across all episodes
        alpha: Learning rate
        gamma: Discount factor
        epsilon_start: Initial exploration rate
        epsilon_end: Final exploration rate
        epsilon_decay_steps: Steps over which to decay epsilon
        max_episode_steps: Maximum steps per episode
        batch_size: Size of mini-batch sampled from replay buffer
        buffer_capacity: Maximum size of replay buffer
        min_buffer_size: Minimum buffer size before training star
ts
        target_update_freq: Update target network every N steps
(hard update)
        frame_stack: Number of frames to stack (N=4 in paper)
        frame_skip: Repeat action for N frames (N=4 in paper)
```

```
        clip_rewards: Clip rewards to [-1, +1]
        render_interval: Render every N steps (None = no renderin
g)
        preprocess_fn: Optional function to preprocess observatio
ns
        use_visual_obs: If True, use env.render() for observation
s
    """
    import copy

    # Create environment
    if use_visual_obs:
        env = gym.make(env_name, render_mode='rgb_array')
    else:
        env = gym.make(env_name)

    # Rendering environment (separate instance)
    env_render = None
    if render_interval is not None:
        if use_visual_obs:
            env_render = gym.make(env_name, render_mode='rgb_arra
y')
        else:
            env_render = gym.make(env_name, render_mode='human')

    n_actions = env.action_space.n

    if render_interval:
        print(f"Rendering: Every {render_interval} steps\n")
    else:
        print("Rendering: Disabled\n")

    # Set random seeds
    np.random.seed(0)
    torch.manual_seed(0)
    random.seed(0)

    # Initialize replay buffer
    replay_buffer = ReplayBuffer(buffer_capacity)

    # Create target network (clone of Q_net)
    Q_target = copy.deepcopy(Q_net)
    Q_target.load_state_dict(Q_net.state_dict())

    # Initialize optimizer and loss
    optimizer = optim.Adam(Q_net.parameters(), lr=alpha)
    criterion = nn.MSELoss()

    # Statistics
    stats = {'episode_returns': [], 'episode_lengths': [], 'cumul
ative_steps': []}

    # Helper function for epsilon annealing
    def get_epsilon(step):
        if step >= epsilon_decay_steps:
            return epsilon_end
        return epsilon_start - (epsilon_start - epsilon_end) * (s
```

```
tep / epsilon_decay_steps)

    # Helper function for frame stacking
    def stack_frames(frame_stack_deque, frame, is_new_episode):
        if is_new_episode:
            # Reset and fill with same frame
            frame_stack_deque.clear()
            for _ in range(frame_stack):
                frame_stack_deque.append(frame)
        else:
            # Add new frame
            frame_stack_deque.append(frame)
        # Stack frames into single array
        return np.concatenate(list(frame_stack_deque), axis=0)

    # Initialize frame stack deque
    from collections import deque
    frame_stack_deque = deque(maxlen=frame_stack)

    total_steps = 0
    episode = 0

    while total_steps < max_total_steps:
        # Use render environment at intervals
        use_render = (render_interval is not None and
                      total_steps % render_interval < max_episode_
steps)
        current_env = env_render if use_render else env

        # Reset environment
        current_env.reset()
        if use_visual_obs:
            frame = current_env.render()
        else:
            frame, _ = current_env.reset()
        if preprocess_fn is not None:
            frame = preprocess_fn(frame)

        # Initialize frame stack for new episode
        state = stack_frames(frame_stack_deque, frame, is_new_epi
sode=True)

        episode_reward = 0
        episode_steps = 0
        done = False

        while not done and episode_steps < max_episode_steps and
total_steps < max_total_steps:
            # Get current epsilon value (annealed)
            epsilon_current = get_epsilon(total_steps)

            # Epsilon-greedy action selection (using online Q_ne
t)
            if np.random.rand() < epsilon_current:
                action = np.random.choice(n_actions)
            else:
                state_tensor = torch.FloatTensor(state).unsqueeze
```

```
(0)
                with torch.no_grad():
                    action = torch.argmax(Q_net(state_tensor)).it
em()

            # Frame skipping: repeat action for frame_skip frames
            total_reward = 0
            for skip_idx in range(frame_skip):
                next_frame_raw, reward, done, truncated, info = c
urrent_env.step(action)
                # Reward clipping
                if clip_rewards:
                    reward = np.clip(reward, -1.0, 1.0)
                total_reward += reward

                # Get and preprocess frame
                if use_visual_obs:
                    next_frame = current_env.render()
                else:
                    next_frame = next_frame_raw
                if preprocess_fn is not None:
                    next_frame = preprocess_fn(next_frame)

                # Stack this frame (captures motion across skippe
d frames)
                next_state = stack_frames(frame_stack_deque, next
_frame, is_new_episode=False)

                if done or truncated:
                    break

            episode_reward += total_reward
            episode_steps += frame_skip
            total_steps += frame_skip  # Count all skipped frames

            if truncated:
                done = True

            # Store transition in replay buffer
            replay_buffer.push(state, action, total_reward, next_
state, done)

            # Train if buffer has enough samples
            if len(replay_buffer) >= min_buffer_size:
                # Sample mini-batch from replay buffer
                states, actions, rewards, next_states, dones = re
play_buffer.sample(batch_size)

                # Convert to tensors
                states_tensor = torch.FloatTensor(states)
                actions_tensor = torch.LongTensor(actions).unsque
eze(1)
                rewards_tensor = torch.FloatTensor(rewards).unsqu
eeze(1)
                next_states_tensor = torch.FloatTensor(next_state
s)
                dones_tensor = torch.FloatTensor(dones).unsqueeze
```

```
(1)

                # Current Q-values (from online network)
                q_values = Q_net(states_tensor).gather(1, actions
_tensor)

                # Target Q-values (using target network - KEY DIF
FERENCE!)
                with torch.no_grad():
                    next_q_values = Q_target(next_states_tensor)
                    max_next_q_values = next_q_values.max(1, keep
dim=True)[0]
                    targets = rewards_tensor + gamma * max_next_q
_values * (1 - dones_tensor)

                # Compute loss and update online network
                loss = criterion(q_values, targets)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                # Hard update target network every target_update_
freq steps
                if total_steps % target_update_freq == 0:
                    Q_target.load_state_dict(Q_net.state_dict())

            state = next_state
            # Print progress
            if render_interval is not None and (total_steps % ren
der_interval == 0):
                recent_episodes = min(10, len(stats['episode_retu
rns']))
                avg_return = np.mean(stats['episode_returns'][-re
cent_episodes:])
                epsilon_current = get_epsilon(total_steps)
                print(f"Steps: {total_steps}/{max_total_steps}, E
pisodes: {episode}, Avg Return (last {recent_episodes}): {avg_ret
urn:.1f}, ε: {epsilon_current:.3f}")

        # Record statistics
        stats['episode_returns'].append(episode_reward)
        stats['episode_lengths'].append(episode_steps)
        stats['cumulative_steps'].append(total_steps)
        episode += 1

    env.close()
    if env_render is not None:
        env_render.close()

    return q_net_replay_buffer_target_net, stats
```

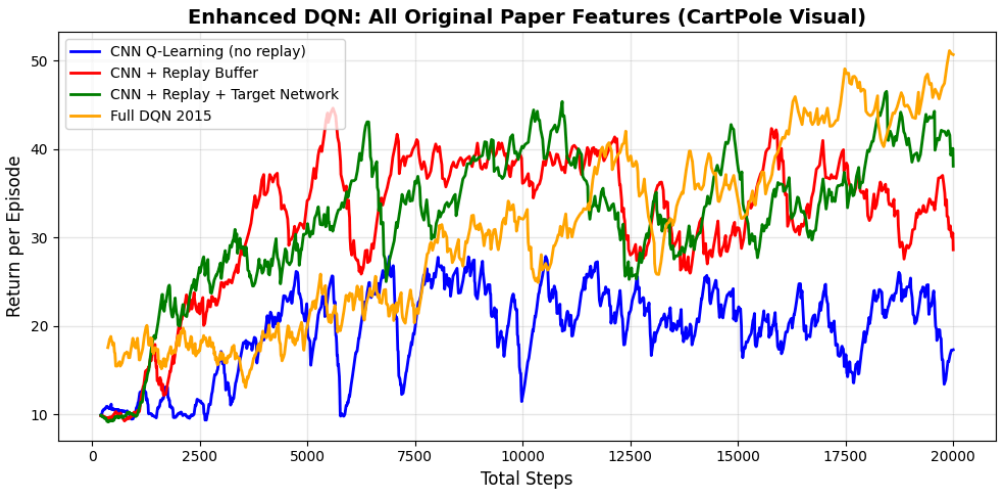
```
============================================================
🎮 DQN with Visual Observations (Replay + Target Network)
============================================================
```

In [13]:
```
# Train Enhanced DQN with all four improvements
from helper_functions import preprocess_image

print("Training Enhanced DQN with all DQN paper features...")
print("Features: Frame Stacking (4) + Frame Skipping (4) + Reward
Clipping + ε-Annealing")

aggregate_frames = 2

# Create CNN Q-network for visual observations (4 stacked graysca
le frames)
q_net_enhanced = QNetworkCNN(
    n_actions=2,
    input_channels=aggregate_frames,
    img_height=42,
    img_width=42
)

q_net_enhanced, stats_enhanced = dqn_2015(
    q_net_enhanced,
    env_name='CartPole-v1',
    max_total_steps=20000,
    alpha=0.001,
    gamma=0.99,
    epsilon_start=1.0,
    epsilon_end=0.1,
    epsilon_decay_steps=10000,
    max_episode_steps=100,
    batch_size=32,
    min_buffer_size=1000,
    target_update_freq=100,
    frame_stack=aggregate_frames,
    frame_skip=aggregate_frames,
    clip_rewards=True,
    render_interval=1000,
    preprocess_fn=preprocess_image,  # Add preprocessing to conve
rt RGB to grayscale 42x42
    use_visual_obs=True
)


# Plot results
plot_continuous_learning_curve(
    [q_net_stats, q_net_replay_buffer_stats, q_net_replay_buffer_
target_net_stats, stats_enhanced],
    window=20,
    labels=['CNN Q-Learning (no replay)', 'CNN + Replay Buffer',
'CNN + Replay + Target Network', 'Full DQN 2015'],
    title="Enhanced DQN: All Original Paper Features (CartPole Vi
sual)"
)

print(f"\\nFinal average return (last 50 episodes): {np.mean(stat
s_enhanced['episode_returns'][-50:]):.1f}")
print("\\nFeatures implemented:")
print("✓ Frame Stacking: 4 consecutive frames as input")
```

```
print("✓ Frame Skipping: Action repeated for 4 frames")
print("✓ Reward Clipping: Rewards clipped to [-1, +1]")
print("✓ ε-Annealing: Linear decay from 1.0 → 0.1")
print("✓ Experience Replay + Target Network")
```

```
Training Enhanced DQN with all DQN paper features...
Features: Frame Stacking (4) + Frame Skipping (4) + Reward Clippi
ng + ε-Annealing
Rendering: Every 1000 steps

Steps: 1000/20000, Episodes: 56, Avg Return (last 10): 16.2, ε:
0.910
Steps: 1000/20000, Episodes: 56, Avg Return (last 10): 16.2, ε:
0.910
Steps: 2000/20000, Episodes: 112, Avg Return (last 10): 19.7, ε:
0.820
Steps: 2000/20000, Episodes: 112, Avg Return (last 10): 19.7, ε:
0.820
Steps: 3000/20000, Episodes: 168, Avg Return (last 10): 17.0, ε:
0.730
Steps: 3000/20000, Episodes: 168, Avg Return (last 10): 17.0, ε:
0.730
Steps: 4000/20000, Episodes: 226, Avg Return (last 10): 19.8, ε:
0.640
Steps: 4000/20000, Episodes: 226, Avg Return (last 10): 19.8, ε:
0.640
Steps: 5000/20000, Episodes: 277, Avg Return (last 10): 23.3, ε:
0.550
Steps: 5000/20000, Episodes: 277, Avg Return (last 10): 23.3, ε:
0.550
Steps: 6000/20000, Episodes: 320, Avg Return (last 10): 26.0, ε:
0.460
Steps: 6000/20000, Episodes: 320, Avg Return (last 10): 26.0, ε:
0.460
Steps: 7000/20000, Episodes: 363, Avg Return (last 10): 23.2, ε:
0.370
Steps: 7000/20000, Episodes: 363, Avg Return (last 10): 23.2, ε:
0.370
Steps: 8000/20000, Episodes: 400, Avg Return (last 10): 36.4, ε:
0.280
Steps: 8000/20000, Episodes: 400, Avg Return (last 10): 36.4, ε:
0.280
Steps: 9000/20000, Episodes: 434, Avg Return (last 10): 28.1, ε:
0.190
Steps: 9000/20000, Episodes: 434, Avg Return (last 10): 28.1, ε:
0.190
Steps: 10000/20000, Episodes: 465, Avg Return (last 10): 28.7, ε:
0.100
Steps: 10000/20000, Episodes: 465, Avg Return (last 10): 28.7, ε:
0.100
Steps: 11000/20000, Episodes: 499, Avg Return (last 10): 30.2, ε:
0.100
Steps: 11000/20000, Episodes: 499, Avg Return (last 10): 30.2, ε:
0.100
Steps: 12000/20000, Episodes: 525, Avg Return (last 10): 42.6, ε:
0.100
Steps: 12000/20000, Episodes: 525, Avg Return (last 10): 42.6, ε:
0.100
Steps: 13000/20000, Episodes: 555, Avg Return (last 10): 29.7, ε:
0.100
Steps: 13000/20000, Episodes: 555, Avg Return (last 10): 29.7, ε:
```

```
0.100
Steps: 14000/20000, Episodes: 583, Avg Return (last 10): 34.6, ε:
0.100
Steps: 14000/20000, Episodes: 583, Avg Return (last 10): 34.6, ε:
0.100
Steps: 15000/20000, Episodes: 610, Avg Return (last 10): 37.0, ε:
0.100
Steps: 15000/20000, Episodes: 610, Avg Return (last 10): 37.0, ε:
0.100
Steps: 16000/20000, Episodes: 636, Avg Return (last 10): 37.8, ε:
0.100
Steps: 16000/20000, Episodes: 636, Avg Return (last 10): 37.8, ε:
0.100
Steps: 17000/20000, Episodes: 659, Avg Return (last 10): 42.0, ε:
0.100
Steps: 17000/20000, Episodes: 659, Avg Return (last 10): 42.0, ε:
0.100
Steps: 18000/20000, Episodes: 681, Avg Return (last 10): 37.6, ε:
0.100
Steps: 18000/20000, Episodes: 681, Avg Return (last 10): 37.6, ε:
0.100
Steps: 19000/20000, Episodes: 703, Avg Return (last 10): 44.3, ε:
0.100
Steps: 19000/20000, Episodes: 703, Avg Return (last 10): 44.3, ε:
0.100
Steps: 20000/20000, Episodes: 722, Avg Return (last 10): 50.7, ε:
0.100
Steps: 20000/20000, Episodes: 722, Avg Return (last 10): 50.7, ε:
0.100
```




```
\nFinal average return (last 50 episodes): 46.2
\nFeatures implemented:
✓ Frame Stacking: 4 consecutive frames as input
✓ Frame Skipping: Action repeated for 4 frames
✓ Reward Clipping: Rewards clipped to [-1, +1]
✓ ε-Annealing: Linear decay from 1.0 → 0.1
✓ Experience Replay + Target Network
```

## Case Study: DQN on Atari Games

### DQN on Atari: Superhuman Performance

First deep RL system to achieve human-level control from raw pixels on multiple Atari games.

(Levine et al., CS285 Lecture 8, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-8.pdf))

**Architecture:**

- Input: 84×84 grayscale images (4 stacked frames)
- 3 Convolutional layers for feature extraction
- 2 Fully connected layers
- Output: Q-values for each action

**Key Innovations:**

- Experience replay (1M transitions)
- Target network (updated every 10k steps)
- Frame stacking (captures temporal information)
- Reward clipping ([-1, +1] normalization)

**Performance:**

- Trained on 49 Atari games
- Human-level performance on 29 games
- Superhuman on several (Breakout, Pong)

(Eysenbach, COS435 Lecture 11: Deep Q Learning, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-11-dqn.pdf))

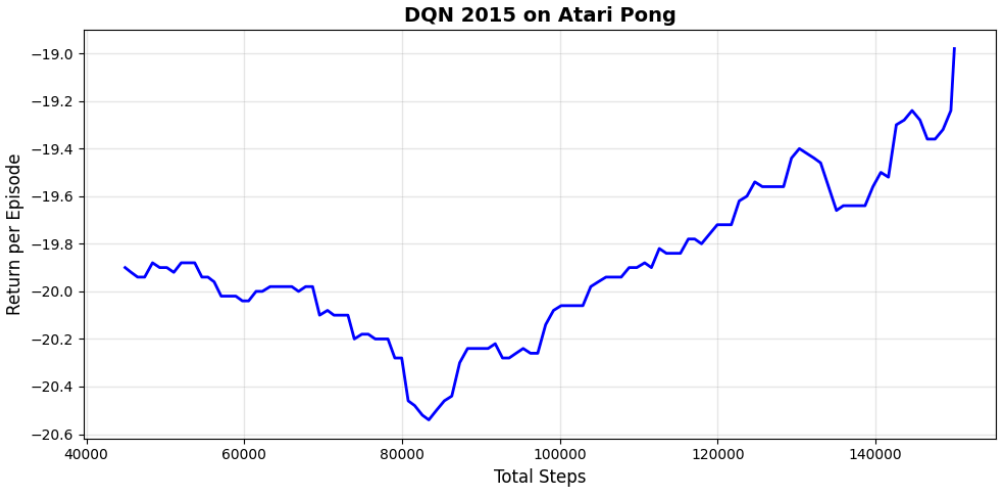
In [14]:
```
# Train DQN on Atari Pong (fast-training game)
print("=" * 60)
print("🎮 DQN 2015 on Atari Pong")
print("=" * 60)

# Import and register ALE (Atari Learning Environment)
import ale_py
import gymnasium
gymnasium.register_envs(ale_py)

# Pong is one of the faster Atari games to train
# It has a simple 2-action space (up/down) and clear reward signa
l

# Create CNN Q-network for Atari (4 stacked grayscale 84x84 frame
s)
q_net_atari = QNetworkCNN(
    n_actions=6,  # Pong has 6 discrete actions in ALE
    input_channels=4,  # 4 stacked frames
    img_height=84,  # Standard Atari preprocessing size
    img_width=84
)

# Atari preprocessing function (convert to grayscale, resize to 8
4x84)
def preprocess_atari(frame):
    """Preprocess Atari frame: RGB to grayscale, resize to 84x84,
normalize"""
    import cv2
    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
    # Resize to 84x84
    resized = cv2.resize(gray, (84, 84), interpolation=cv2.INTER_
AREA)
    # Normalize to [0, 1]
    normalized = resized / 255.0
    # Add channel dimension
    return normalized[np.newaxis, ...]  # Shape: (1, 84, 84)

print("Training DQN on Pong-v5...")
print("Note: This will take longer than CartPole. Reduced steps f
or demo purposes.")

q_net_atari, stats_atari = dqn_2015(
    q_net_atari,
    env_name='ALE/Pong-v5',  # Atari Pong
    max_total_steps=150000,  # Reduced from typical 10M for demo
    alpha=0.00025,  # Lower learning rate for Atari (as in paper)
    gamma=0.99,
    epsilon_start=1.0,
    epsilon_end=0.1,
    epsilon_decay_steps=100000,  # Anneal over most of training
    max_episode_steps=1000,  # Pong episodes can be longer
    batch_size=32,
    buffer_capacity=50000,  # Larger buffer for Atari
    min_buffer_size=10000,  # More burn-in
```

```
    target_update_freq=1000,  # Update target less frequently
    frame_stack=4,
    frame_skip=4,
    clip_rewards=True,
    render_interval=5000,
    preprocess_fn=preprocess_atari,
    use_visual_obs=True
)

# Plot results
plot_continuous_learning_curve(
    stats_atari,
    window=50,
    title="DQN 2015 on Atari Pong"
)

print(f"\nFinal average return (last 20 episodes): {np.mean(stats
_atari['episode_returns'][-20:]):.1f}")
print("\nNote: Full Atari training requires millions of steps. Th
is is a short demo.")
```

```
============================================================
🎮 DQN 2015 on Atari Pong
============================================================
Training DQN on Pong-v5...
Note: This will take longer than CartPole. Reduced steps for demo
purposes.
Training DQN on Pong-v5...
Note: This will take longer than CartPole. Reduced steps for demo
purposes.
Rendering: Every 5000 steps

Rendering: Every 5000 steps

Steps: 5000/150000, Episodes: 5, Avg Return (last 5): -20.4, ε:
0.955
Steps: 5000/150000, Episodes: 5, Avg Return (last 5): -20.4, ε:
0.955
Steps: 10000/150000, Episodes: 11, Avg Return (last 10): -20.2,
ε: 0.910
Steps: 10000/150000, Episodes: 11, Avg Return (last 10): -20.2,
ε: 0.910
Steps: 15000/150000, Episodes: 16, Avg Return (last 10): -19.6,
ε: 0.865
Steps: 15000/150000, Episodes: 16, Avg Return (last 10): -19.6,
ε: 0.865
Steps: 20000/150000, Episodes: 22, Avg Return (last 10): -20.1,
ε: 0.820
Steps: 20000/150000, Episodes: 22, Avg Return (last 10): -20.1,
ε: 0.820
Steps: 25000/150000, Episodes: 28, Avg Return (last 10): -20.1,
ε: 0.775
Steps: 25000/150000, Episodes: 28, Avg Return (last 10): -20.1,
ε: 0.775
Steps: 30000/150000, Episodes: 33, Avg Return (last 10): -19.4,
ε: 0.730
Steps: 30000/150000, Episodes: 33, Avg Return (last 10): -19.4,
ε: 0.730
Steps: 35000/150000, Episodes: 38, Avg Return (last 10): -20.2,
ε: 0.685
Steps: 35000/150000, Episodes: 38, Avg Return (last 10): -20.2,
ε: 0.685
Steps: 40000/150000, Episodes: 44, Avg Return (last 10): -19.1,
ε: 0.640
Steps: 40000/150000, Episodes: 44, Avg Return (last 10): -19.1,
ε: 0.640
Steps: 45000/150000, Episodes: 50, Avg Return (last 10): -19.5,
ε: 0.595
Steps: 45000/150000, Episodes: 50, Avg Return (last 10): -19.5,
ε: 0.595
Steps: 50000/150000, Episodes: 55, Avg Return (last 10): -20.5,
ε: 0.550
Steps: 50000/150000, Episodes: 55, Avg Return (last 10): -20.5,
ε: 0.550
Steps: 55000/150000, Episodes: 61, Avg Return (last 10): -20.3,
ε: 0.505
Steps: 55000/150000, Episodes: 61, Avg Return (last 10): -20.3,
```

```
ε: 0.505
Steps: 60000/150000, Episodes: 67, Avg Return (last 10): -20.2,
ε: 0.460
Steps: 60000/150000, Episodes: 67, Avg Return (last 10): -20.2,
ε: 0.460
Steps: 65000/150000, Episodes: 72, Avg Return (last 10): -20.3,
ε: 0.415
Steps: 65000/150000, Episodes: 72, Avg Return (last 10): -20.3,
ε: 0.415
Steps: 70000/150000, Episodes: 78, Avg Return (last 10): -20.4,
ε: 0.370
Steps: 70000/150000, Episodes: 78, Avg Return (last 10): -20.4,
ε: 0.370
Steps: 75000/150000, Episodes: 84, Avg Return (last 10): -20.5,
ε: 0.325
Steps: 75000/150000, Episodes: 84, Avg Return (last 10): -20.5,
ε: 0.325
Steps: 80000/150000, Episodes: 90, Avg Return (last 10): -20.7,
ε: 0.280
Steps: 80000/150000, Episodes: 90, Avg Return (last 10): -20.7,
ε: 0.280
Steps: 85000/150000, Episodes: 95, Avg Return (last 10): -20.7,
ε: 0.235
Steps: 85000/150000, Episodes: 95, Avg Return (last 10): -20.7,
ε: 0.235
Steps: 90000/150000, Episodes: 101, Avg Return (last 10): -19.3,
ε: 0.190
Steps: 90000/150000, Episodes: 101, Avg Return (last 10): -19.3,
ε: 0.190
Steps: 95000/150000, Episodes: 106, Avg Return (last 10): -19.5,
ε: 0.145
Steps: 95000/150000, Episodes: 106, Avg Return (last 10): -19.5,
ε: 0.145
Steps: 100000/150000, Episodes: 111, Avg Return (last 10): -19.5,
ε: 0.100
Steps: 100000/150000, Episodes: 111, Avg Return (last 10): -19.5,
ε: 0.100
Steps: 105000/150000, Episodes: 117, Avg Return (last 10): -18.8,
ε: 0.100
Steps: 105000/150000, Episodes: 117, Avg Return (last 10): -18.8,
ε: 0.100
Steps: 110000/150000, Episodes: 122, Avg Return (last 10): -19.5,
ε: 0.100
Steps: 110000/150000, Episodes: 122, Avg Return (last 10): -19.5,
ε: 0.100
Steps: 115000/150000, Episodes: 127, Avg Return (last 10): -19.8,
ε: 0.100
Steps: 115000/150000, Episodes: 127, Avg Return (last 10): -19.8,
ε: 0.100
Steps: 120000/150000, Episodes: 133, Avg Return (last 10): -19.7,
ε: 0.100
Steps: 120000/150000, Episodes: 133, Avg Return (last 10): -19.7,
ε: 0.100
Steps: 125000/150000, Episodes: 138, Avg Return (last 10): -19.2,
ε: 0.100
Steps: 125000/150000, Episodes: 138, Avg Return (last 10): -19.2,
ε: 0.100
```

```
Steps: 130000/150000, Episodes: 143, Avg Return (last 10): -19.4,
ε: 0.100
Steps: 130000/150000, Episodes: 143, Avg Return (last 10): -19.4,
ε: 0.100
Steps: 135000/150000, Episodes: 148, Avg Return (last 10): -19.8,
ε: 0.100
Steps: 135000/150000, Episodes: 148, Avg Return (last 10): -19.8,
ε: 0.100
Steps: 140000/150000, Episodes: 154, Avg Return (last 10): -20.0,
ε: 0.100
Steps: 140000/150000, Episodes: 154, Avg Return (last 10): -20.0,
ε: 0.100
Steps: 145000/150000, Episodes: 159, Avg Return (last 10): -18.4,
ε: 0.100
Steps: 145000/150000, Episodes: 159, Avg Return (last 10): -18.4,
ε: 0.100
Steps: 150000/150000, Episodes: 164, Avg Return (last 10): -17.9,
ε: 0.100
Steps: 150000/150000, Episodes: 164, Avg Return (last 10): -17.9,
ε: 0.100
```




```
Final average return (last 20 episodes): -18.4

Note: Full Atari training requires millions of steps. This is a s
hort demo.
```

## Some Notes on Convergence

The progression from tabular to deep Q-learning comes with **significant theoretical trade-offs**:

### ✓ Tabular Q-Learning: Convergence Guaranteed

**Theorem (Watkins & Dayan, 1992)**: Tabular Q-learning converges to $Q^*(s, a)$ with probability 1 if:

1. All state-action pairs $(s, a)$ are visited infinitely often
2. Step-size $\alpha_n(s, a)$ satisfies Robbins-Monro conditions: $\sum_n \alpha_n = \infty$ and $\sum_n \alpha_n^2 < \infty$
3. Rewards are bounded

**Key Property**: Converges off-policy without requiring environment model.

### ✓ Linear Function Approximation: Convergence Guaranteed for on-policy (approximate SARSA) but not off-policy (Q-Learning)

**Theorem (Tsitsiklis & Van Roy, 1997)**: TD(0) with linear function approximation $\hat{V}(s, w) = w^T x(s)$ converges to the TD fixed point $w_{\text{TD}}$ under on-policy distribution, with error bound:

$$\text{VE}(w_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_w \text{VE}(w)$$

where **VE (Mean Squared Value Error)** measures approximation quality:

$$\text{VE}(w) = \sum_{s \in \mathcal{S}} \mu(s) \Big[ V^\pi(s) - \hat{V}(s, w) \Big]^2$$

**Interpretation:**

- $V^\pi(s)$: True value function under policy $\pi$
- $\hat{V}(s, w)$: Approximate value function with parameters $w$
- $\mu(s)$: On-policy state distribution (how often we visit state $s$)
- $\min_w \text{VE}(w)$: Best possible approximation with linear features
- $\frac{1}{1-\gamma}$: Expansion factor - TD's solution is within this factor of optimal

The multiplier $\frac{1}{1-\gamma}$ is typically small (e.g., $= 10$ for $\gamma = 0.9$, meaning TD finds a solution within 10× of the best possible linear approximation).

### ⚠️ The Deadly Triad: Divergence Risk

**Sutton & Barto (2018, Section 11.3)** identify three elements that together can cause instability:

1. **Function approximation** (especially non-linear)
2. **Bootstrapping** (TD-style updates using estimates)
3. **Off-policy learning** (behavior policy $b(a|s) \neq$ target policy $\pi(a|s)$)

### ❌ DQN: No Convergence Guarantees

**Critical Finding**: Deep Q-Networks have **NO formal convergence proof**. As Sutton & Barto note:

> "There are counterexamples showing divergence for Q-learning [with function approximation]. This is cause for concern because otherwise Q-learning has the best convergence guarantees of all control methods." (Section 11.3, p. 264)

***Baird's Counterexample (1995)***

A simple 7-state MDP demonstrating divergence with **linear function approximation**:

**Setup:**

- 6 "upper" states ($s_1, \ldots, s_6$) + 1 "lower" state ($s_7$) — named from Baird's original diagram layout
- **Reward function**: $r(s, a, s') = 0$ for all transitions (no immediate rewards)
- **Target policy** $\pi$: Always goes to $s_7$
- **Behavior policy** $b$: Stays in upper states (uniform transitions)
- Linear approximation: $\hat{V}(s, w) = w^T x(s)$ with shared features

**Note:** The MDP itself only defines states, transitions, and rewards. The parameters $w$ belong to the **learning algorithm** (TD with function approximation), not the MDP. We write $\hat{V}(s, w)$ to emphasize that the value estimate depends on the current weights $w$ being learned.

**Why It Diverges:**

The deadly triad creates a vicious feedback loop:

1. Behavior policy $b$ keeps agent in upper states, never visiting $s_7$
2. But we're learning values for target policy $\pi$ (which would go to $s_7$)
3. If $\hat{V}(s_7, w)$ accidentally increases → TD targets for upper states increase → pushes upper state values up → which further increases $\hat{V}(s_7, w)$ (shared features)
4. Feedback loop: $\hat{V}(s_7) \uparrow \Rightarrow \text{targets} \uparrow \Rightarrow \hat{V}(\text{upper}) \uparrow \Rightarrow \hat{V}(s_7) \uparrow\uparrow \rightarrow \infty$

**Key Insight:** Estimates become **unanchored** because we never actually visit $s_7$ under behavior policy $b$, yet use our own inflated estimates as bootstrapped targets. With on-policy learning (behavior = target), updates would be grounded in actual experience.

**DQN's Success**: Works through **engineering solutions** rather than theoretical guarantees:

- Experience replay: Smooths data distribution, reduces correlations
- Target networks: Stabilizes learning targets
- Architecture choices: Gradient clipping, network design

As Mnih et al. (2015) state, DQN provides "human-level control through deep reinforcement learning" via empirical success on 49 Atari games, but convergence remains **empirically validated, not theoretically proven**.

> "The potential for off-policy learning remains tantalizing, the best way to achieve it still a mystery." — Sutton & Barto (2018, Section 11.10)

# Summary

## What We've Implemented: Progressive DQN Variations

In this lecture, we've built a complete understanding of deep Q-learning through progressive implementation:

**1. Basic Neural Q-Learning** ( `q_learning_with_network` )

- Q-learning with neural network function approximation
- Accepts any network architecture (MLP, CNN)
- Direct on-policy learning from environment interaction
- **Issue**: Highly correlated consecutive samples → unstable learning

**2. Experience Replay** ( `q_learning_with_replay` )

- Added replay buffer $\mathcal{D}$ storing $(s, a, r, s')$ transitions
- Random minibatch $\mathcal{B}$ sampling breaks temporal correlations
- Data efficiency: each experience used in multiple updates
- **Improvement**: More stable learning, but still moving target problem

**3. Target Networks** ( `q_learning_with_replay_and_target` )

- Separate target network $Q(s, a; \theta^-)$ updated every $C$ steps
- Loss: $\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$
- **Improvement**: Reduced oscillations, more stable convergence

**4. Full DQN 2015** ( `dqn_2015` )

- All previous features plus:
  - **Frame Stacking** ($N = 4$ frames): Captures temporal dynamics/motion
  - **Frame Skipping** ($N = 4$ actions): Computational efficiency, action persistence
  - **Reward Clipping** to $[-1, +1]$: Normalized learning signals across tasks
  - **Epsilon Annealing**: Linear decay from $\epsilon_{\text{start}}$ to $\epsilon_{\text{end}}$
- **Result**: Human-level performance on Atari games (Mnih et al., 2015)

## Limitations of Q-Learning

While DQN is powerful for discrete action spaces (Atari games, board games), it has fundamental limitations:

1. **Discrete Actions Only**: Q-learning requires computing $\max_{a \in \mathcal{A}} Q(s, a)$, which is:
    - Feasible when actions are discrete (e.g., $|\mathcal{A}| = 18$ Atari actions)
    - Infeasible when actions are continuous (e.g., $\mathcal{A} \subseteq \mathbb{R}^d$ robot joint angles)
    - Requires discretization, losing precision
2. **High-Dimensional Action Spaces**: Even with discrete actions, if the action space is large (e.g., $|\mathcal{A}| \approx 10^3$ actions), finding the maximum becomes expensive
3. **Indirect Policy Learning**: Q-learning learns values, then derives policy via $\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a)$. For some tasks, learning the policy directly is more natural.
4. **No Convergence Guarantees with Non-Linear Approximation**: Neural networks work empirically but lack theoretical safety nets

## Outlook to next lecture: Combining with Q-Learning with Policy Gradients - Actor-Critic

Recall from **Lecture 3** we learned to directly optimize policies using the policy gradient theorem. By combining:

- **Value-based methods** (this lecture): TD learning for state/action evaluation
- **Policy-based methods** (Lecture 3): Direct policy optimization via gradients

We arrive at **Actor-Critic architectures**:

- **Critic**: Value network $Q_\phi(s, a)$ with parameters $\phi$, learned via TD methods (what we know now)
- **Actor**: Policy network $\pi_\theta(a|s)$ with parameters $\theta$, updated via policy gradients (from Lecture 3)

**Deep Deterministic Policy Gradient (DDPG)** combines DQN's function approximation with deterministic policy gradients, enabling continuous control:

- Actor network $\pi_\theta(s)$ for continuous action selection (deterministic policy)
- Critic network $Q_\phi(s, a)$ (like DQN) for value estimation
- Experience replay buffer $\mathcal{D}$ and target networks from DQN

This opens the door to robotic control, autonomous driving, and other continuous control domains that DQN cannot handle.